An Exponential Succinctness Gap between Three-Variable Logic and the Calculus of Relations

Yuya Uezato
CyberAgent, Inc.
uezato_yuya@cyberagent.co.jp

Yoshiki Nakamura
Chiba University
nakamura.yoshiki.ny@gmail.com

Abstract

Three-variable first-order logic (FO^3) and the calculus of relations define the same binary queries, an equivalence that goes back to Tarski's work in the 1940s. The classical translation from FO^3 into the calculus is single-exponential, but it has remained open whether this blowup is necessary. We prove that it is, even over finite structures and even when the translated terms may share subterms. Our proof introduces a new technique: we modify one finite structure in exponentially many small ways that each change the answer of a short FO^3 formula. In any equivalent term, only compositions of binary relations can notice these modifications, and each composition can notice at most two. The same technique gives an exponential succinctness gap between FO^3 and FO^2 sentences, and exponential lower bounds for matrix query languages, where the analogous question was also open.

1 Introduction

A classical result of Tarski from the 1940s states that *three-variable first-order logic* (FO^3) and the *calculus of relations* (CoR) [Tar41], a variable-free algebra of binary relations, express the same binary queries [TG87, Giv06]. The known translations between them have asymmetric size bounds:

$$\varphi(x,y) \in \mathrm{FO}^3 \xrightarrow{|t| = 2^{O(|\varphi|)}} t(x,y) \in \mathrm{CoR}, \qquad t(x,y) \in \mathrm{CoR} \xrightarrow{|\varphi| = O(|t|)} \varphi(x,y) \in \mathrm{FO}^3.$$

Whether this exponential blowup is unavoidable has remained unknown [FGL+15, Nak22, NU27]. **We show that the exponential blowup from FO^3 to CoR is unavoidable**. Before stating our result precisely in §1.1, we review the importance of FO^3 and this translation in logic and computation.

Expressive power of FO^3. FO^3 is singular among the finite-variable fragments FO^ℓ. FO^3 can express the transitivity law $\forall x \forall y \forall z\, (E(x,y) \wedge E(y,z) \to E(x,z))$, and thereby, together with irreflexivity and the absence of maximal elements, force infinite models. In contrast, FO^2 cannot express transitivity and has the finite model property [Mor75]. Over words, the variable hierarchy of FO collapses at three: every FO-definable property of words can be expressed in FO^3 [IK89], whereas FO^2 is strictly less expressive [EVW02]. Remarkably, three variables even suffice as a foundation for mathematics: Tarski and Givant showed that set theories such as ZFC can be formalized by CoR equations [TG87, Giv06, Mad21]. This expressive strength is also reflected in computational complexity. For FO^3, satisfiability and finite satisfiability are undecidable, since already the $\forall\exists\forall$ prenex class is undecidable [KMW62, BGG97]; undecidability holds even over a single binary relation [Nak19]; for FO^2, both are decidable [Mor75]. For a fixed sentence, model counting can be $\#\mathrm{P}_1$-complete in FO^3 [BVGS15] but takes polynomial time in FO^2 [VMD14]. Asser's longstanding question of whether first-order spectra (i.e., the sets of finite-model sizes) are closed under complement [Ass55] is equivalent to the open problem NE = coNE [JS74]. Resolving it for FO^3 over a single binary relation would already settle the general problem [KT15, KT18].

Algorithmic power of the FO^3*-to-*CoR *translation.* The preceding results illustrate the logical and computational power of FO^3. Its exceptional status persists even in model checking. For fixed ℓ, a sentence $\varphi \in \mathrm{FO}^\ell$ can be evaluated on an n-element structure in $O(|\varphi| n^\ell)$ time [Var95]. For $\ell \geq 4$, this exponent is conditionally optimal under the hyperclique hypothesis [BS21], [NU27, Corollary 32]. For FO^3, however, the cubic barrier can be broken: FO^3 model checking takes $2^{O(|\varphi|)} n^\omega$ time [NU27, Theorem 10], where $\omega < 2.3714$ is the exponent of matrix multiplication [ADV+25]. **The key to subcubic FO^3 model checking is the classical FO^3-to-CoR translation**. It transforms a formula φ into a CoR term t, which can be evaluated in $O(|t| n^\omega)$ time because a composition is a Boolean matrix product and every other operation takes $O(n^2)$ time. Thus, the translation size directly affects the running time. If every FO^3 formula admitted a polynomial-size CoR representation, every such query would have a compact algebraic evaluation plan, executable in $\mathrm{poly}(|\varphi|) n^\omega$ time once compiled. Our result rules out this possibility: some FO^3 queries inherently require exponentially large CoR representations, even with shared subterms.

1.1 Main results

CoR *terms* denote binary relations on a nonempty domain $\mathbb{D}$, built from relation symbols, the constants $\mathbf{0} := \varnothing$, $\mathbf{1} := \mathbb{D}^2$, and $\mathbf{id} := \{(u,u) : u \in \mathbb{D}\}$, using union $X \cup Y$, intersection $X \cap Y$,

complement $\overline{X} \coloneqq \mathbb{D}^2 \setminus X$ (also written $\neg X$), converse $X^\smile \coloneqq \{(v,u) : (u,v) \in X\}$, and composition $X;Y$, defined by

$$X;Y \coloneqq \{(u,v) : \text{some } w \in \mathbb{D} \text{ satisfies } (u,w) \in X \text{ and } (w,v) \in Y\}.$$

Composition implements the elimination step $\exists z\,(P(x,z) \wedge Q(z,y)) \equiv (P;Q)(x,y)$; a vertex z with $P(x,z)$ and $Q(z,y)$ is a *witness* for (x,y).

A CoR term T is a syntax tree with $|T|$ nodes and $\#_;(T)$ compositions. A CoR *circuit* C is a DAG (rather than a tree) and may share subterms; its total size $|C|$ and $g_;(C)$ count its distinct nodes and composition gates. For an FO formula φ, we write $T \equiv_{\text{fin}} \varphi$ or $C \equiv_{\text{fin}} \varphi$ for equivalence on all finite structures over the input signature, identifying formulas, terms, and circuits with the binary queries they define.

Our hard family and its translation into CoR **(upper bound).** We use the following single-quantifier positive FO^3 formula over $2k$ binary relation symbols $P_1, \ldots, P_k, Q_1, \ldots, Q_k$:

$$G_k(x,y) \coloneqq \exists z \bigwedge_{i=1}^{k} \Big[(P_i(x,y) \wedge Q_i(x,z)) \vee (Q_i(x,y) \wedge P_i(x,z))\Big].$$

The signature of G_k consists only of binary relations, but grows with k: $|G_k| = O(k)$. It asks whether the queried pair (x,y), the *read edge*, *accepts* some *witness edge* (x,z): at every coordinate $i \in \{1,\ldots,k\}$, one of the two edges satisfies P_i and the other Q_i. The choice may differ across coordinates, but the same witness z must serve all of them.

The natural way to translate G_k into CoR is to expand it into disjunctive normal form (DNF), choosing one of the two disjuncts at each coordinate. Let $[k] \coloneqq \{1,\ldots,k\}$. For $A \subseteq [k]$, let $A^c \coloneqq [k] \setminus A$ and $H_A \coloneqq (\bigcap_{i\in A} P_i) \cap (\bigcap_{i\notin A} Q_i)$, where an empty intersection is $\mathbf{1}$.

$$G_k(x,y) \equiv \exists z \bigvee_{A\subseteq[k]} \Big[\bigwedge_{i\in A}(P_i(x,y) \wedge Q_i(x,z)) \wedge \bigwedge_{i\notin A}(Q_i(x,y) \wedge P_i(x,z))\Big] \equiv \exists z \bigvee_{A\subseteq[k]} [H_A(x,y) \wedge H_{A^c}(x,z)]$$

$$\equiv \bigvee_{A\subseteq[k]} [H_A(x,y) \wedge \exists z\, H_{A^c}(x,z)] \equiv \Big(\bigcup_{A\subseteq[k]} [H_A \cap (H_{A^c};\mathbf{1})]\Big)(x,y).$$

The last step uses row projection: $(X;\mathbf{1})(x,y) \equiv \exists z\, X(x,z)$. The DNF yields a CoR term T_k with 2^k compositions and $O(k2^k)$ nodes. By sharing partial intersections among the H_A, it also yields a CoR circuit C_k with 2^k composition gates and only $O(2^k)$ nodes. Thus, we have $\#_;(T_k) = g_;(C_k) = 2^k$, $|T_k| = O(k2^k)$, and $|C_k| = O(2^k)$.

The same idea underlies the classical translation of arbitrary FO^3 formulas into CoR: DNF expansion allows quantifiers to be eliminated using relational composition.

Proposition (Classical upper bound; see Appendix D)**.** *Every* FO^3 *formula* φ *over binary relations with at most two free variables has an equivalent* CoR *term, and hence a circuit, of total size* $2^{O(|\varphi|)}$.

Main result: lower bound. One might hope that the DNF expansion can be substantially compressed using more sophisticated CoR terms. Surprisingly, the straightforward translation is already optimal within a factor of two in the number of compositions.

Theorem 1.1 (Exponential lower bounds)**.** *For every* $k \geq 1$*, every* CoR *circuit* $C \equiv_{\text{fin}} G_k$ *has at least* 2^{k-1} *composition gates, and every* CoR *term* $T \equiv_{\text{fin}} G_k$ *has at least* 2^{k-1} *compositions.*

It counts only compositions: all other operations are unrestricted and free. Since $|G_k| = O(k)$, the theorem establishes an unconditional exponential succinctness gap from FO^3 to CoR, even when the target is a circuit rather than a term. Together with the classical single-exponential upper bound, this settles the worst-case size of the FO^3-to-CoR translation.

The gap is not an artifact of free variables: the FO^3 sentence $G_k^{\forall} := \forall x \forall y \, (x = y \vee G_k(x, y))$ has size $O(k)$, yet every equivalent CoR equation requires at least $2^{k-1} - 1$ compositions, even with shared intermediate gates (Corollary 4.2). Thus, compiling FO^3 sentences into CoR equations for model checking also entails an unavoidable exponential blowup in expression size.

Domain size. Theorem 1.1 concerns circuits $C \equiv_{\mathrm{fin}} G_k$, which compute G_k on all finite structures. How large must the domain be before the exponential lower bound becomes unavoidable? We show that exponentially large domains already suffice, even for circuits specialized to a fixed domain size.

Proposition 1.2 (Domain size)**.** *For some constant $c > 0$, for every $k \geq 1$ and $n \geq ck4^k$, every* CoR *circuit computing G_k on all n-element structures has at least 2^{k-1} composition gates.*

In contrast, for $n < 2^k/(8k)$, Proposition 4.4 gives circuits with only $4kn \, (< 2^{k-1})$ compositions that compute G_k on all structures with at most n elements. Whether the domain-size bound $ck4^k$ is asymptotically optimal remains open.

Over a single binary relation. How large is the succinctness gap from FO^3 to CoR over a single binary relation? By encoding the $2k$ input relations of G_k into one binary relation, with only a logarithmic factor increase in formula size, we obtain tight **subexponential** bounds.

Theorem 1.3 (Subexponential bounds; see Theorem 5.1)**.** *Over a single binary relation, every* FO^3 *query $\varphi(x, y)$, with $|\varphi|$ its syntax-tree size, has an equivalent* CoR *circuit of size $2^{O(|\varphi|/\log|\varphi|)}$ (upper bound), and some* FO^3 *queries $\psi(x, y)$ require $2^{\Omega(|\psi|/\log|\psi|)}$ composition gates (lower bound).*

Whether a corresponding subexponential upper bound holds for CoR terms remains open.

Model-checking consequence. The constructive upper bound yields a $2^{O(m/\log m)} n^{\omega+\eta}$-time algorithm for model checking FO^3 sentences of size m on n-vertex directed graphs, for every fixed $\eta > 0$ (see §5.3). This improves the previous $2^{O(m)} n^{\omega+\eta}$-time algorithm [NU27, Theorem 10]. Moreover, under the low-dimensional 3-OV hypothesis, the $m/\log m$ exponent is conditionally optimal for truly subcubic algorithms [NU27, Theorem 72].

1.2 Limits of the relational fooling-set argument

A natural approach is to extend the earlier exponential lower bound for Σ_1CoR, even with shared subterms [Nak22, Theorems 6.1 and 6.12]. Here Σ_1CoR allows complement only on input relations and **id** (after pushing it to the leaves): $\overline{R}; \mathbf{1}$ is permitted, but $\overline{R; \mathbf{1}}$ is not. Although the original proof in [Nak22] uses formula-size games in the style of Adler and Immerman [AI03], it can be viewed as a relational fooling-set argument. We first recall the fooling-set argument.

The *fooling-set argument* [Yao79, LS81, KN97, RY20] is a standard technique for proving lower bounds in communication complexity. For a Boolean function $f \colon A \times B \to \{0, 1\}$, a *fooling set* is a set $F \subseteq f^{-1}(1)$ such that any two distinct $(a, b), (a', b') \in F$ have a crossed input (a, b') or (a', b) of

value 0. A *1-rectangle* is a product $A' \times B' \subseteq f^{-1}(1)$. Since a rectangle containing two points also contains their crossings, each 1-rectangle covers at most one point of F, so at least $|F|$ are needed.

Consider Nakamura's query of size $O(k)$, $\varphi_k(x,y) \coloneqq \exists z \bigwedge_{i=1}^{k}(R_i(x,z) \vee R_i(z,y))$ [Nak22]. Let $I, J \subseteq [k]$ record the relation labels on the two witness edges (x,z) and (z,y). The witness condition is precisely $f_k(I,J) = 1 \iff I \cup J = [k]$, with the fooling set $F_k = \{(I, I^c) : \varnothing \subsetneq I \subsetneq [k]\}$ of size $2^k - 2$ for $k \geq 2$. Each point (I, I^c) is realized in [Nak22, §5] by a positive pointed structure $(\mathcal{S}_I; u_I, v_I)$ with a witness path $u_I \xrightarrow{I} w_I \xrightarrow{I^c} v_I$. There is a subtlety: these paths belong to different structures, so we cannot directly cross (u_I, v_I) and (u_J, v_J). The construction provides a common negative pointed structure $(\mathcal{N}; u^*, v^*)$, with $\mathcal{S}_I \models \varphi_k(u_I, v_I)$ for every I but $\mathcal{N} \not\models \varphi_k(u^*, v^*)$. Whenever $f_k(I, J^c) = 0$, the left witness half (u_I, w_I) and the right half (w_J, v_J) embed into $\mathcal{N}$ at $(u^*, c_{I,J})$ and $(c_{I,J}, v^*)$, respectively, using a vertex $c_{I,J}$.

How can we cross witnesses from different structures? The key is *preservation under embeddings*: every Σ_1CoR term V and embedding $\iota \colon \mathcal{A} \hookrightarrow \mathcal{B}$ satisfy

$$\iota([\![V]\!]_{\mathcal{A}}) \subseteq [\![V]\!]_{\mathcal{B}}, \quad \text{or equivalently,} \quad \mathcal{A} \models V(u,v) \implies \mathcal{B} \models V(\iota(u), \iota(v)). \tag{$\diamond$}$$

Now let T be a Σ_1CoR term equivalent to φ_k. A short structural induction on T (after pushing converses to the leaves, and using the embeddings to exclude endpoint witnesses) shows that, for each I, some composition subterm $X;Y$ of T satisfies (1) $\mathcal{S}_I \models X(u_I, w_I) \wedge Y(w_I, v_I)$; and (2) $\mathcal{N} \not\models (X;Y)(u^*, v^*)$. Suppose $\mathcal{S}_J \models X(u_J, w_J) \wedge Y(w_J, v_J)$ also holds for $I \neq J$. Since $(I, I^c), (J, J^c) \in F_k$, at least one of their crossed inputs has value 0. Suppose $f_k(I, J^c) = 0$; the other case is symmetric. The crossing embeddings and ($\diamond$) then give $\mathcal{N} \models (X;Y)(u^*, v^*)$ because $\mathcal{N} \models X(u^*, c_{I,J}) \wedge Y(c_{I,J}, v^*)$. This contradicts (2).

Thus, **such composition $X;Y$ plays the role of a 1-rectangle**, accounting for at most one fooling-set point. The argument extends to shared subterms [Nak22, §6.3].

Why complement defeats the argument. The crossing argument relies crucially on ($\diamond$), which fails under unrestricted complement. Indeed, $V := \overline{R;\mathbf{1}}$ violates ($\diamond$). $V(x,y)$ means $\neg\exists z.R(x,z)$ in FO^3. Let $\mathcal{A}$ have no R-successor of u, and obtain an extension $\mathcal{B}$ by adding a fresh R-successor. Then, $\mathcal{A} \models V(u,v)$ and $\mathcal{B} \not\models V(u,v)$.

Furthermore, the above $2^k - 2$ positive instances $(\mathcal{S}_I; u_I, v_I)$ and the negative instance $(\mathcal{N}; u^*, v^*)$ can be distinguished by a single full-CoR term with only two compositions. Writing $E := \bigcup_i (R_i \cup R_i^\smile)$, consider $U := \neg((E \cap (E; \neg\mathbf{id})); \mathbf{1})$ where $\neg T$ means $\overline{T}$. $U(x,y)$ means $\neg[\exists y \exists z.(E(x,y) \wedge E(x,z) \wedge y \neq z)]$ in FO^3. This holds in every $\mathcal{S}_I$ but fails in $\mathcal{N}$, where u^* has at least two neighbors. Although U does not define φ_k, the old test instances themselves cannot certify an exponential lower bound for full CoR.

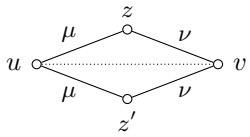
1.3 Technical overview

Rather than extending the fooling-set argument in §1.2, we abandon witness crossing and embeddings altogether. Instead, we compare a reference structure with its local modifications using exact predictions that commute with complement.

Our technique: The two-witness property. We use a complete undirected graph $\mathbb{G} = (V, \chi)$ whose edges are colored by the 2^k subsets of $[k]$. We write $\chi(u,v) = \chi(v,u)$ for the color of the edge between distinct vertices u and v, and use λ, μ, ν for colors. The key property of $\mathbb{G}$ is the

two-witness property: for all distinct $u, v \in V$ and all colors μ, ν,

at least two vertices $z \notin \{u, v\}$ satisfy $\chi(u, z) = \mu$ and $\chi(z, v) = \nu$.



We call such vertices z *witnesses*. A coloring with this property exists whenever $|V| \geq n_0(2^k)$, where $n_0(N) := \lceil 12N^2 \ln(2N) \rceil + 3$, since a uniformly random coloring has it with positive probability (Lemma 3.1). These colorings go back to Jipsen, Maddux, and Tuza [JMT95].

Roadmap. In §1.3.1, we construct 2^k folds of the reference structure $\mathcal{M}_0$, each changing the answer of G_k at an unchanged edge. In §1.3.2, we show that every circuit gate on $\mathcal{M}_0$ is constant on each edge-color class and on the diagonal. This allows us to define predictions under each fold. Boolean operations, including complement, and converse preserve these predictions; only composition can violate them. In §1.3.3, we show that, thanks to the two-witness property, each composition can violate the prediction for at most two folds. Since every fold must be detected, any circuit for G_k requires at least 2^{k-1} compositions, even with sharing.

1.3.1 Folding the reference structure

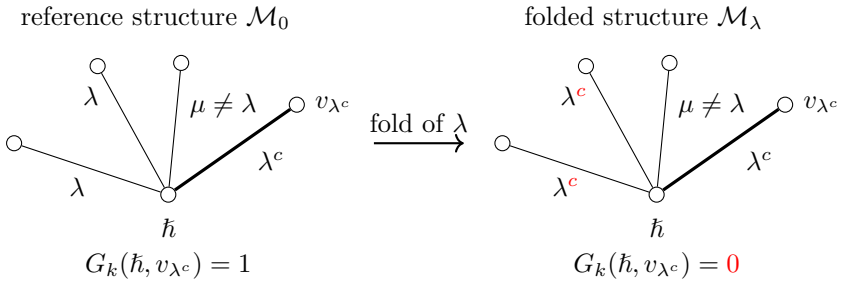
We use the reference coloring χ to define an input structure $\mathcal{M}_0$ for G_k. On an edge of color μ, we interpret P_i as true exactly when $i \in \mu$, and Q_i exactly when $i \notin \mu$. No input relation holds on loops, which carry the separate label **id**. Consequently, a read edge of color μ accepts exactly witness edges of color μ^c: for distinct u, v,

$$\mathcal{M}_0 \models G_k(u, v) \iff \exists z \notin \{u, v\}.\ \chi(u, z) = \chi(u, v)^c.$$

By the two-witness property, every vertex has edges of every color, so G_k holds at every non-loop pair of $\mathcal{M}_0$.

Fix a vertex $\hbar \in V$, the *hub*, and for each color μ choose v_μ with $\chi(\hbar, v_\mu) = \mu$. For each color λ, let χ_λ be obtained from χ by recoloring every λ-colored edge incident to $\hbar$ as λ^c, leaving all other edges unchanged. Let $\mathcal{M}_\lambda$ be the input structure induced by χ_λ in the same way as $\mathcal{M}_0$. In $\mathcal{M}_\lambda$, the hub has no edge of color λ, but the read edge $(\hbar, v_{\lambda^c})$ remains unchanged (Figure 1).

Figure 1: The fold of λ. Only edges at the hub $\hbar$ are drawn.



The read edge $(\hbar, v_{\lambda^c})$ requires a λ-colored edge at $\hbar$; the fold removes all such edges. Hence

$$\chi_\lambda(\hbar, v_{\lambda^c}) = \chi(\hbar, v_{\lambda^c}) = \lambda^c,\ \mathcal{M}_0 \models G_k(\hbar, v_{\lambda^c}),\ \mathcal{M}_\lambda \not\models G_k(\hbar, v_{\lambda^c}).$$

We call $(\mathcal{M}_0, \mathcal{M}_\lambda, (\hbar, v_{\lambda^c}))$ the *test* for λ. Thus G_k yields 2^k tests, one for each color.

1.3.2 Circuits see only colors

Color-determined relations. A relation $X \subseteq V^2$ is *color-determined* if $\chi(u,v) = \chi(u',v') \implies X(u,v) = X(u',v')$ holds. Here loops carry the separate label **id**. Equivalently, X is a union of edge-color classes, possibly together with the diagonal. Write $X[\mu]$ for its common value on edges of color μ, $X[\mathbf{id}]$ for its value on loops.

Every gate of an arbitrary CoR circuit computes a color-determined relation on $\mathcal{M}_0$ (Lemma 3.2). We can prove this by induction on the circuit gates. Input relations and constants are color-determined, while Boolean operations and converse preserve this property. The only nontrivial step is composition $X;Y$.

For distinct u, v, the two-witness property realizes every pair of colors μ, ν along a path $u \xrightarrow{\mu} z \xrightarrow{\nu} v$ with $z \notin \{u, v\}$. The case $z \in \{u, v\}$ depends only on $\chi(u,v)$ and the diagonal values of X, Y. Hence $(X;Y)(u,v)$ depends only on $\chi(u,v)$. For a loop (u,u), both edges of a path $u - z - u$ have the same color, and every color occurs at every vertex. Together with the case $z = u$, this makes $(X;Y)(u,u)$ independent of u, completing the induction.

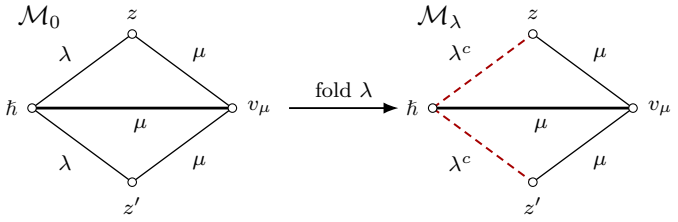
Predicting and noticing a fold. A color-determined relation X assigns a value $X[\mu]$ to each color μ. To predict the effect of a fold, we keep these values fixed and read them at the new edge colors:

$$\textit{(Prediction)} \quad X^{(\lambda)}(u,v) := X[\chi_\lambda(u,v)].$$

Thus a recolored edge reads $X[\lambda^c]$ rather than $X[\lambda]$; all other pairs retain their values. This predicts what a gate would compute if its color-dependent behavior on $\mathcal{M}_0$ remained unchanged after the fold; its actual value on $\mathcal{M}_\lambda$ may differ.

Input relations and constants behave exactly as predicted. Moreover, Boolean operations and converse commute with prediction (Proposition 3.3); in particular, $\overline{X^{(\lambda)}} = (\overline{X})^{(\lambda)}$. Unlike the embedding inclusion ($\diamond$) of §1.2, this equality survives unrestricted complement.

Figure 2 compares three operations for any two distinct colors λ, μ: X selects the λ-colored edges, and Y the μ-colored edges (both exclude loops). The thick edge $e = (\hbar, v_\mu)$ is unchanged by the fold of λ, which removes all $X^{(\lambda)}$-edges at $\hbar$.



Operation	Predicted	Actual	Notice?
$X;Y$	1	0	Yes
$X \cup Y$	1	1	No
$\overline{X}$	1	1	No

Figure 2: At $e = (\hbar, v_\mu)$, only composition notices the fold of λ. Only two witness paths are shown; all λ-edges at $\hbar$ are recolored to λ^c.

An operation *notices* a fold at given color-determined operands if applying it to their predictions differs from predicting its result. In Figure 2, composition notices the fold at e, since

$$(X^{(\lambda)};Y^{(\lambda)})(e) = 0 \neq 1 = (X;Y)^{(\lambda)}(e).$$

By contrast, union and complement do not notice the fold. More generally, no Boolean operation or converse can notice any fold (Proposition 3.3).

Every fold must be noticed by some composition gate. Now let C be any circuit computing G_k. If no composition gate notices the fold of λ, Lemma 3.4 implies that the output on $\mathcal{M}_\lambda$ is the prediction of the output on $\mathcal{M}_0$. Since the test edge $e_\lambda = (\hbar, v_{\lambda^c})$ is unchanged,

$$[\![C]\!]_{\mathcal{M}_\lambda}(e_\lambda) = \left([\![C]\!]_{\mathcal{M}_0}\right)^{(\lambda)}(e_\lambda) = [\![C]\!]_{\mathcal{M}_0}(e_\lambda),$$

contradicting the test for λ, whose outputs are 0 on $\mathcal{M}_\lambda$ and 1 on $\mathcal{M}_0$.

1.3.3 Each composition notices at most two folds

We now show that any composition $X;Y$ of color-determined relations notices at most two folds (Lemma 3.5). The two-witness property first rules out noticing at pairs away from the hub.

Away from the hub (Figure 3). At pairs (u, v) with $u, v \neq \hbar$, composition agrees with prediction. Figure 3 uses the same coloring, operands X, Y, and fold as Figure 2, but observes $e' = (u, v_\mu)$.

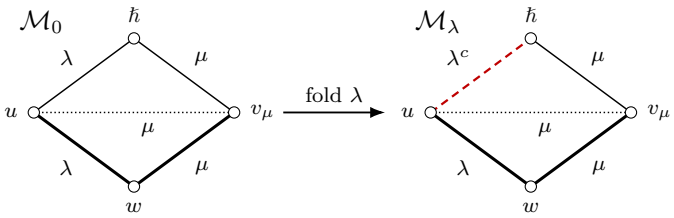


Figure 3: Away from the hub, the fold invalidates the path through $\hbar$ but leaves the second witness path through w unchanged. Only these two witness paths are shown.

In $\mathcal{M}_0$, both paths have colors (λ, μ). After the fold, the path through $\hbar$ is invalidated but the path through w survives. Hence: $(X^{(\lambda)}; Y^{(\lambda)})(e') = 1 = (X;Y)^{(\lambda)}(e')$.

More generally, between any distinct $u, v \neq \hbar$, the two-witness property preserves every two-color path pattern via a witness other than $\hbar$. Loops away from $\hbar$ retain edges of every color, and endpoint witnesses are unchanged. Thus composition cannot violate prediction away from the hub.

At the hub (Figure 2 revisited). Only pairs involving $\hbar$ remain. In Figure 2, X holds only on λ and Y only on μ, where $\lambda \neq \mu$. Their composition notices the folds of λ and μ at $(\hbar, v_\mu)$ and $(v_\lambda, \hbar)$, respectively.

Why can it not notice a third fold? To notice λ at $(\hbar, v)$, X must hold on λ and no other off-diagonal color. Indeed, if $X[\nu] = 1$ for some $\nu \neq \lambda$ and $Y[\tau] = 1$ for some color τ, the two-witness property supplies a path $\hbar \xrightarrow{\nu} z \xrightarrow{\tau} v$ that survives the fold of λ. Thus z remains a witness for $(X^{(\lambda)}; Y^{(\lambda)})(\hbar, v)$. Likewise, noticing a distinct fold μ at $(u, \hbar)$ requires Y to hold only on μ. Hence noticing two distinct folds forces X and Y to be supported on $\{\lambda\}$ and $\{\mu\}$, respectively. No third color occurs in either support, and their supports are disjoint, ruling out an additional fold noticed at $(\hbar, \hbar)$. If X and Y share a color instead, all possible noticed folds must coincide. Thus $X;Y$ notices at most two folds.

Summary. Every one of the 2^k folds requires a noticing composition gate (§1.3.2), whereas each distinct composition gate notices at most two (Lemma 3.5). Hence $2^k \le 2g_{;}(C)$, proving the lower bound of Theorem 1.1 even with sharing.

1.4 Further results

In §1.3, we explained how folds of a reference structure prove our main theorem. We now use this approach to establish further exponential succinctness gaps: from FO^3 to FO^2, from CoR to FO^2, and from three-attribute relational algebra to matrix query languages. To explain these extensions, we first summarize the counting argument of §1.3.

Recall that the folds in §1.3 are indexed by colors $\lambda \in \Lambda = 2^{[k]}$. For a circuit $C \equiv_{\mathrm{fin}} G_k$, let $D_g \subseteq \Lambda$ be the set of colors whose folds are noticed by gate g. Every color belongs to some D_g: otherwise, the circuit could not detect the change at the read edge left unchanged by that fold (§1.3.2). Thus $\bigcup_{g \in C} D_g = \Lambda$, and counting gives

$$2^k \le \sum_{g \in C} |D_g|. \tag{†}$$

For CoR, Boolean operations and converse notice no folds, while each composition notices at most two (§1.3.3). Hence $\sum_{g \in C} |D_g| \le 2g_{;}(C)$, recovering Theorem 1.1. The applications below use the same counting argument, with different bounds on the folds noticed by the target operations and, when needed, a different reference structure.

FO^3 to FO^2. The same query G_k is also FO^2-definable: the DNF expansion in §1.1 gives

$$G_k(x,y) \equiv \bigvee_{A \subseteq [k]} \left[H_A(x,y) \wedge \exists y\, H_{A^c}(x,y) \right]$$

where H_A is FO^2-definable. Here the quantifier binds y only in the second conjunct.

Theorem 1.4. *For every $k \ge 1$, every FO^2 formula equivalent to $G_k(x,y)$ over finite structures has at least 2^k quantifiers.*

The displayed expansion uses exactly 2^k quantifiers, so the bound is tight. To our knowledge, this is the first exponential succinctness gap from FO^3 to FO^2 over general finite structures. The gap also holds for sentences. On pure finite linear orders, by contrast, FO^3 sentences admit polynomial-size FO^2 translations [GS05], while Vinall-Smeeth's exponential separations over general structures start at three target variables [VS24] (see §1.5).

The same reference structure $\mathcal{M}_0$ and folds from §1.3 suffice. Indeed, regard each FO^2 subformula ψ as a binary relation X_ψ of assignments to (x,y), with any unused coordinate treated as a dummy. Quantifying a variable is then a special case of relational composition:

$$X_{\exists y\, \psi} = X_\psi ; \mathbf{1}, \qquad X_{\exists x\, \psi} = \mathbf{1} ; X_\psi .$$

Thus their assignment relations remain color-determined on $\mathcal{M}_0$, just like the CoR gate values in §1.3. Since one operand of each composition is $\mathbf{1}$, each quantifier can notice at most one fold (§1.3.3), rather than the two possible for general composition. Atomic formulas and Boolean connectives

notice none; universal quantifiers are handled by complement. The counting inequality (†) therefore requires at least 2^k quantifiers. We also obtain a lower bound of $k2^k$ relation-atom occurrences and optimal tree size $\Theta(k2^k)$, for both queries and sentences (§6.3).

For CoR itself, we also determine the optimal tree size of G_k as $(2+o(1))k2^k$, whereas its optimal circuit size is $\Theta(2^k)$ (Theorem 6.3).

CoR to FO^2. An exponential succinctness gap also occurs from CoR to FO^2. We need a different query, since G_k already requires exponentially many CoR compositions. Put $\mathscr{P}_i^1 \coloneqq \mathbf{id} \cap P_i$ and $\mathscr{P}_i^0 \coloneqq \mathbf{id} \cap \overline{P_i}$, and let $\vartheta_A(u) \coloneqq \bigwedge_{i\in A} P_i(u,u) \wedge \bigwedge_{i\notin A} \neg P_i(u,u)$ for $A \subseteq [k]$. Consider the following linear-size CoR term and its FO^2 expansion:

$$\mathrm{CoR} : U_k \coloneqq R; \bigcap_{i=1}^{k} \left[(\mathscr{P}_i^0; \mathbf{1}; \mathscr{P}_i^0) \cup (\mathscr{P}_i^1; \mathbf{1}; \mathscr{P}_i^1) \right] \longrightarrow \mathrm{FO}^2 : \bigvee_{A\subseteq[k]} \left[\vartheta_A(y) \wedge \exists y \left(R(x,y) \wedge \vartheta_A(y) \right) \right]$$

Thus U_k asks whether x has an R-successor with the same k-bit loop pattern as y.

Proposition 1.5. *For every $k \geq 1$, every FO^2 formula equivalent to U_k over finite structures has at least 2^k quantifiers.*

The displayed expansion uses exactly 2^k quantifiers, so the bound is tight. For the lower bound, take a new reference structure $\mathcal{W}_0$ with several vertices of each loop pattern and a hub h having R-successors of every pattern (Figure 4). The fold of $A \subseteq [k]$ deletes all R-edges from h to vertices of pattern A. Thus $U_k(h, y_A)$ changes from true to false for a vertex y_A of pattern A, even when (h, y_A) is unchanged.

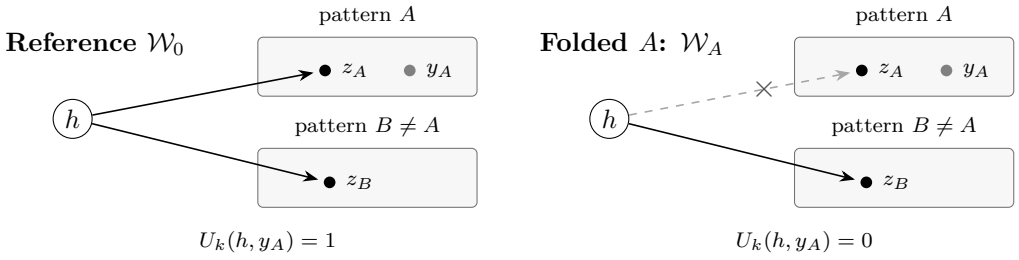


Figure 4: The fold of A removes all R-edges from h to vertices of loop pattern A (schematic).

The original folds cannot give this separation: on their reference structure, every composition notices at most two folds, ruling out a short CoR term that distinguishes all those tests. In the new structure, the outer composition of U_k can notice all 2^k folds, whereas each FO^2 quantifier, viewed as a projection, still notices at most one. The latter bound uses redundant representatives of each atomic pair type. Applying the same counting argument (†) to the new tests proves the lower bound (§6.4).

Arity elimination in matrix query languages. Brijder et al. compared two formalisms for semiring-valued queries: ARA(3), which allows intermediate tables with up to three columns, and MATLANG, which computes with matrices, vectors, and scalars. They showed that the two have the same expressive power when inputs and outputs have at most two columns [BGV22].

Their translation from ARA(3) to MATLANG can grow exponentially, but whether this blowup is unavoidable was left open [BGV19, p. 9]. We establish an exponential lower bound using a semiring-valued analogue of our hard query G_k.

Let $\mathfrak{S} = (S, \oplus, \otimes, 0_S, 1_S)$ be a commutative semiring, with input relations $P_i, Q_i : V^2 \to S$. Define

$$\mathcal{G}_k(x, y) \coloneqq \bigoplus_{z \in V} \bigotimes_{i=1}^{k} \Big[(P_i(x, y) \otimes Q_i(x, z)) \oplus (Q_i(x, y) \otimes P_i(x, z)) \Big].$$

This query has an ARA(3) expression of size $O(k)$: all k conditions are combined over (x, y, z) before z is aggregated away. Over Boolean annotations ($S = \mathbb{B}$, with $\oplus = \vee$ and $\otimes = \wedge$), $\mathcal{G}_k$ specializes to G_k.

In MATLANG, the usual semiring matrix–vector (mv) and matrix–matrix (mm) products take the form

$$(A \cdot v)(x) \coloneqq \bigoplus_{z \in V} A(x, z) \otimes v(z) \quad \text{(mv)}, \qquad (A \cdot B)(x, y) \coloneqq \bigoplus_{z \in V} A(x, z) \otimes B(z, y) \quad \text{(mm)}.$$

Both eliminate their aggregated index immediately. The DNF of §1.1 also gives an explicit MATLANG representation of $\mathcal{G}_k$. Here H_A is obtained from the expression in §1.1 by replacing intersections with pointwise $\otimes$, and $\mathbf{1}_S$ is the all-1_S column vector. Then

$$\mathcal{G}_k(x, y) = \bigoplus_{A \subseteq [k]} H_A(x, y) \otimes (H_{A^c} \cdot \mathbf{1}_S)(x).$$

This uses 2^k mv products, together with row broadcasting and entrywise operations; more general computations may also use mm products and shared intermediate results. For a circuit C, let $\#_{\mathrm{mv}}(C)$ count matrix–vector and vector–matrix aggregations, and $\#_{\mathrm{mm}}(C)$ count matrix–matrix aggregations, with shared gates counted once. All other permitted operations are free.

For a concrete non-Boolean case, consider the natural and integer min–plus semirings $\mathsf{Trop}_D = (D \cup \{+\infty\}, \min, +, +\infty, 0)$, where $D \in \{\mathbb{N}, \mathbb{Z}\}$.

Theorem 1.6 (Tropical arity-elimination overhead)**.** *For every $k \geq 1$ and $\mathfrak{S} \in \{\mathsf{Trop}_{\mathbb{N}}, \mathsf{Trop}_{\mathbb{Z}}\}$, the query $\mathcal{G}_k$ has an ARA(3) expression of size $O(k)$, but every typed matrix circuit C over $\mathfrak{S}$ computing it on all finite inputs satisfies $\#_{\mathrm{mv}}(C) + 3\#_{\mathrm{mm}}(C) \geq 2^k$.*

The lower bound holds even with arbitrary uniform entrywise functions and aggregation kernels. Its proof uses the fold-counting principle (†) (§7).

The argument extends uniformly beyond tropical matrices. Examples include the Boolean semiring $(\{0, 1\}, \vee, \wedge)$ and the max–min semiring $([0, 1], \max, \min)$. It also covers integer max–plus $(\mathbb{Z} \cup \{-\infty\}, \max, +)$ and, for each fixed $b \geq 1$, finite-set annotations $(2^{[b]}, \cup, \cap)$ and b-dimensional min–plus $\mathsf{Trop}_{\mathbb{Z}}^b$. One property covers all these examples: *finite join breadth*, meaning that every finite join is determined by a bounded number of its summands. It bounds how many folds an aggregation can notice, giving exponential lower bounds for every fixed nontrivial commutative idempotent semiring of finite join breadth (Theorem 7.3). Over Boolean matrices, we also obtain the sharper bound $\#_{\mathrm{mv}}(C) + 2\#_{\mathrm{mm}}(C) \geq 2^k$. Positive computations over nonidempotent semirings such as $(\mathbb{N}, +, \times)$ are handled separately by Booleanization (§7.5).

Robustness: closure and randomization. The lower bound persists even when CoR is extended with reflexive–transitive closure $X^* \coloneqq \bigcup_{m \geq 0} X^m$, which can be evaluated in $O(n^\omega)$ time [FM71, Mun71, Fur70]. Each closure gate notices at most one fold.

Proposition 1.7 (Closure does not help)**.** *Let C be a* CoR *circuit with closure gates $(-)^*$ and let $g_*(C)$ count its distinct closure gates. If $C \equiv_{\mathrm{fin}} G_k$, then $2g_{;}(C) + g_*(C) \geq 2^k$.*

The lower bound also extends to bounded-error randomized circuits, even when circuits are specialized to each sufficiently large domain size.

Corollary 1.8 (Fixed sizes and bounded error)**.** *For some constant $c > 0$, for every $k \geq 1$, $n \geq ck4^k$, and $0 \leq \varepsilon < 1/2$, every input-independent distribution of size-specific* CoR *circuits whose output at each pair (u, v) is correct with probability at least $1 - \varepsilon$ on every n-element input satisfies*

$$\mathbb{E}[g_{;}(C)] \geq (1 - 2\varepsilon)2^{k-1}.$$

1.5 Related work

The calculus of relations: history and expressiveness. The calculus of relations (CoR) predates first-order logic as a formal system. De Morgan introduced relational composition and converse in his work on the syllogism in the 1850s and 1860s [DM56, DM64]; Peirce developed the logic of relatives [Pei70, Pei80], and Schröder devoted a volume to it in 1895 [Sch95]. This was before first-order logic was isolated in Hilbert's lectures and Hilbert and Ackermann's textbook [ES13, HA28, Moo88]. The central expressiveness problem was whether every quantified statement about relations could be *condensed* into this variable-free calculus. Schröder claimed that it could [Sch95, p. 551], but Korselt found a counterexample, published by Löwenheim in 1915 [Löw15, Löw67]. In modern terms, the FO^4 sentence $\forall a_1\, a_2\, a_3\, a_4.\, \bigvee_{i,j;i\neq j} a_i = a_j$ states that the domain has at most three elements, but is expressible neither in FO^3 nor by a CoR equation.

Tarski raised the expressiveness question in 1941 [Tar41] and, according to Tarski and Givant, soon obtained the three-variable characterization: CoR defines exactly the FO^3-definable binary queries. A full proof appeared only in their 1987 monograph [TG87]; see also [Mad06]. For historical accounts, see [Pra92, Mad91, BL21, Bra00]. Their original recursive translation [TG87, §3.9] did not analyze its size. A straightforward bottom-up translation using DNF conversion [HA28, Kle52, Sch89] and quantifier elimination by relational composition gives a single-exponential upper bound (Appendix D); the reverse CoR-to-FO^3 translation is linear.

Relational computation and quantifier elimination. The operations underlying CoR have long served as computational primitives, often in calculi distinct from full CoR. Copilowish gave a matrix-based treatment of the calculus over finite domains [Cop48], while Levien and Maron applied relational operations to retrieval and inference [LM65, LM67, Lev69]. Codd's relational algebra laid the foundations of database querying [Cod70, Cod72], followed by work on relational-expression equivalence, optimization, and efficient join evaluation [ASU79b, ASU79a, SAC+79, Yan81]. Binary relational-expression evaluation became an algorithmic subject [HSU74, HSU77, SU75, SU77], with applications to parsing and recursive database queries [SSS85, GSSS87, GSSS91]. Related algebras also appear in relational programming, temporal reasoning, database execution, and graph querying [Mac81, Mon74, All83, BK99, CMW87, FGL+15, HPV+21, HVGW22].

A second line of work exploits relational composition as existential quantifier elimination, using matrix multiplication for fast evaluation. Early examples include transitive closure [Fur70, FM71, Mun71, OO73, GM93] and algorithms for triangles, cliques, cycles, and small subgraphs [IR78, NP85, AYZ97, KKM00, EG04, YZ04]. Already in 1977, Chandra and Merlin studied generalized

join–project operations and their evaluation by rectangular Boolean matrix multiplication [CM77]. Williams's algorithm for prenex first-order graph properties [Wil14] can be viewed in the same algebraic framework: DNF-based quantifier elimination yields ordinary CoR composition for three quantifiers and typed composition over tuples for larger prenex formulas, evaluated using rectangular matrix multiplication (as in [CM77]). Gao and Impagliazzo [GI19] also treated general FO^3 formulas, beyond prenex normal form, using matrix-based evaluation. Related techniques appear in first-order graph algorithms [BFK19] and database join–project and join–aggregate evaluation [DHK20, HY20, Hu24, AHS25]. Most directly, our earlier work used the FO^3-to-CoR translation for truly subcubic FO^3 model checking [NU27].

Succinctness results. Nakamura proved exponential lower bounds for positive CoR and Σ_1CoR, even for circuits, and a superpolynomial separation over one binary relation [Nak22, §6]. The case of full CoR remained open (see §1.2). In our earlier work [NU27, Corollary 40], we ruled out polynomial-time FO^3-to-CoR translations under the 3-OV hypothesis, but not the existence of small equivalent representations. Theorem 1.1 gives an unconditional exponential lower bound even for CoR circuits, while Theorem 1.3 gives tight subexponential circuit bounds over one binary relation.

On finite linear orders with no additional relations, Grohe and Schweikardt proved a polynomial-size translation from FO^3 to FO^2 sentences [GS05, Theorem 4.3]. Since FO^2 translates linearly into CoR, this also yields polynomial-size CoR equations. By contrast, our exponential succinctness gaps from FO^3 to both FO^2 and CoR hold even for sentences over general finite structures. They also showed that full first-order logic is exponentially more succinct than FO^3 on the same orders, already witnessed by FO^4 [GS05, Theorem 5.3]. The short FO^4 formulas combine divide-and-conquer for distances [Sav70] with the classical quantifier-based abbreviation trick, which avoids duplicating recursive subformulas [FR74, Sto74, FR79, CH90, Vor96, SM02]. The FO^3 lower bound draws on Adler–Immerman formula-size games [AI03]. However, the resulting distance predicates admit linear-size CoR circuits by repeated squaring, $S_{i+1} = S_i; S_i$; our lower bound holds even with such sharing.

Vinall-Smeeth established exponential succinctness gaps from $\mathrm{FO}^{\ell+1}$ to FO^{ℓ} for every $\ell \geq 3$, using higher-arity (non-binary) relations [VS24, Theorem 2]. Our result establishes the FO^3-to-FO^2 case over binary signatures, with an exponential lower bound on the number of quantifiers. The linear FO^2-to-CoR translation was already known [Nak22], while Proposition 1.5 gives an exponential gap in the reverse direction.

Lower-bound methods. As explained in §1.2, the witness-crossing argument underlying Nakamura's formula-size-game proof for Σ_1CoR [AI03, Nak22] relies on embeddings, which fail under unrestricted complement. We also considered other standard lower-bound techniques, but their usual resource measures do not directly fit our setting. Formula-size games [GS05, FvdHIK13, HV19], Ehrenfeucht–Fraïssé and pebble games [Fra54, Ehr61, Imm82, Lib04], and multi-structural games [Imm81, FLRV21, CFI+24] address formula size, quantifier depth, variable count, or quantifier number, rather than the number of distinct composition gates in circuits with sharing. Indeed, G_k has only one quantifier and admits a CoR expression of composition depth one; the issue is the *number* of compositions. Monotone-circuit lower bounds [Raz85, AB87] are natural candidates for our positive query, but equivalent full-CoR circuits may use unrestricted complement. Switching-lemma and random-restriction methods [Hås86] typically exploit shallow Boolean-circuit structure, whereas we impose no restrictions on Boolean nesting and charge only compositions.

Communication complexity suggests further approaches. Rank, discrepancy, and Karchmer–Wigderson techniques [MS82, KN97, KW90, Juk12] do not directly bound compositions when Boolean operations are free. For example, set disjointness has rank 2^k, yet its 1-entries form the complement of a union of only k rectangles. Alternating existential and universal witness tests also resemble the nondeterministic and co-nondeterministic levels of the communication polynomial hierarchy [BFS86, GPW18], but we know of no direct transfer of lower bounds to our circuit model. These approaches did not yield the required composition-count bound.

Our proof instead draws on the affine-plane and random colorings of Jipsen, Maddux, and Tuza [JMT95], which, as explained in the technical overview, already yield the two-witness property needed here. The new ingredient is our fold-based argument: it bounds how many local folds each distinct operation can notice, even with unrestricted complement and circuit sharing.

1.6 Organization

Section 2 fixes semantics and resource measures. Section 3 builds the reference structure, proves that unnoticed folds leave a circuit's values as predicted and that each composition notices at most two folds, and states the test–cost theorem. Section 4 supplies the tests for G_k and proves Theorem 1.1 together with the other bounds for relational circuits: equations, fixed sizes and bounded error, circuits without converse, domain size, random inputs, and closure. Section 5 proves the tight fixed-vocabulary circuit bounds and derives the model-checking application. Section 6 transfers the bound to quantifiers, determines the minimum term size, and proves Theorem 1.4 and Proposition 1.5. Section 7 gives a concrete ARA(3) query, formalizes typed matrix circuits, and extends the fold analysis to semiring-valued aggregation. The appendices give explicit two-witness colorings, the term-size charging argument and the sharpness of the fold tests, the directed construction for circuits without converse, and the classical single-exponential translation.

2 Preliminaries

We fix the semantics of finite-variable formulas, relational terms, and circuits, with examples that will be used throughout the paper. All structures have nonempty domains. Our lower bounds concern finite structures, whereas the explicit translations hold on all nonempty structures.

2.1 Finite-variable logic and binary queries

Signatures and formulas. A *signature* σ is a finite set of binary relation symbols; there are no function or individual constant symbols. A σ-structure $\mathcal{M}$ has domain $D := \mathrm{dom}(\mathcal{M}) \neq \varnothing$ and interprets each $R \in \sigma$ as a relation $R^{\mathcal{M}} \subseteq D^2$. Equality is available. For $\ell \geq 1$, the *ℓ-variable fragment* $\mathrm{FO}^\ell[\sigma]$ consists of first-order formulas using only ℓ variable names, with arbitrary reuse and rebinding of those names. Formally, with u, v ranging over a fixed set of ℓ names, its syntax is

$$\varphi ::= R(u,v) \mid u = v \mid \neg\varphi \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid \exists u\, \varphi \mid \forall u\, \varphi.$$

We use the names x, y for FO^2 and x, y, z for FO^3. The restriction is on the *number of variable names*, not on the number or nesting depth of quantifiers. For example,

$$\varphi(x,y) := P(x,y) \wedge \exists y\, Q(x,y)$$

is an FO^2 formula: the quantifier rebinds y only in its own scope, leaving the occurrence of y in $P(x, y)$ free. Write $\mathrm{FV}(\varphi)$ for the set of free variables.

Assignment relations. A formula with $\mathrm{FV}(\varphi) \subseteq \{x, y\}$ defines the *binary query*

$$[\![\varphi]\!]_{\mathcal{M}} \coloneqq \{(a, b) \in D^2 : \mathcal{M} \models \varphi[x \mapsto a, y \mapsto b]\}.$$

A variable absent from $\mathrm{FV}(\varphi)$ contributes a *dummy coordinate*: the query is independent of its assigned value. Thus a sentence φ has the binary assignment value

$$[\![\varphi]\!]_{\mathcal{M}} = \begin{cases} D^2, & \mathcal{M} \models \varphi, \\ \varnothing, & \mathcal{M} \not\models \varphi. \end{cases}$$

This convention lets formulas, relational terms, and circuits have the same kind of output. An FO^3 subformula may nevertheless have *three* free variables: for example, $\theta(x, y, z) \coloneqq R(x, z) \wedge S(z, y)$ has the ternary assignment relation $\{(a, b, c) \in D^3 : R(a, c) \wedge S(c, b)\}$. Quantifying z returns a binary query. Accordingly, direct FO^3 evaluation can store three-variable truth tables in $O(|\varphi| n^3)$ time on an n-element domain, whereas every intermediate value of a CoR term or circuit is binary. Section 7 revisits this arity contrast for matrix query languages.

2.2 The calculus of relations and relational circuits

Relational terms. The *calculus of relations* (CoR) has terms generated by

$$T ::= R \mid \mathbf{0} \mid \mathbf{1} \mid \mathbf{id} \mid (T \cup T) \mid (T \cap T) \mid \overline{T} \mid T^{\smile} \mid (T; T), \qquad R \in \sigma.$$

Every term denotes a binary relation on D. Its constants are $\mathbf{0} \coloneqq \varnothing$, $\mathbf{1} \coloneqq D^2$, and $\mathbf{id} \coloneqq \{(a, a) : a \in D\}$. Union and intersection are set operations; for relations $X, Y \subseteq D^2$ the other operations are

$$\overline{X} \coloneqq D^2 \setminus X, \quad X^{\smile} \coloneqq \{(b, a) : (a, b) \in X\}, \quad (X; Y) \coloneqq \{(a, b) : \exists c \in D.(X(a, c) \wedge Y(c, b))\}.$$

Here $X(a, b)$ abbreviates $(a, b) \in X$; complement is unrestricted, including around compositions. A *composition* $X; Y$ is Boolean matrix multiplication, with c its existential *witness*.

Two useful special cases are the *row and column projections*:

$$\begin{aligned} (X; \mathbf{1})(a, b) &\iff \exists c \in D\ X(a, c), \\ (\mathbf{1}; X)(a, b) &\iff \exists c \in D\ X(c, b). \end{aligned} \tag{Proj}$$

The first is independent of b, and the second of a. More generally, for binary predicates A, B, C with the indicated variable pairs,

$$\exists z\, (A(x, y) \wedge B(x, z) \wedge C(z, y)) \quad \equiv \quad (A \cap (B; C))(x, y). \tag{QE}$$

For example, the FO^2 query $P(x, y) \wedge \exists y\, Q(x, y)$ above is $P \cap (Q; \mathbf{1})$ in CoR. The position of complement matters: $\overline{R; \mathbf{1}}$ tests that x has no R-successor, whereas $\overline{R}; \mathbf{1}$ tests that x has some non-R pair. These queries are not equivalent.

A linear translation from FO^2. Under the assignment convention above, the standard translation τ from FO^2 into CoR is given on atoms and quantifiers by

φ	$\tau(\varphi)$	φ	$\tau(\varphi)$
$R(x,y)$	R	$R(y,x)$	$R^\smile$
$R(x,x)$	$(R \cap \mathbf{id}); \mathbf{1}$	$R(y,y)$	$\mathbf{1}; (R \cap \mathbf{id})$
$x = y$ or $y = x$	$\mathbf{id}$	$x = x$ or $y = y$	$\mathbf{1}$
$\exists x\, \psi$	$\mathbf{1}; \tau(\psi)$	$\exists y\, \psi$	$\tau(\psi); \mathbf{1}$

Conjunction, disjunction, and negation become intersection, union, and complement; universal quantifiers use $\forall u\, \psi \equiv \neg\exists u\, \neg\psi$. Structural induction, using (Proj), shows that

$$[\![\tau(\varphi)]\!]_{\mathcal{M}} = [\![\varphi]\!]_{\mathcal{M}}, \qquad |\tau(\varphi)| = O(|\varphi|).$$

The translation also preserves the number of relation-atom occurrences, including reflexive atoms. See [Nak22, Remark 3.22]. Section 6 bounds the fold cost of this translation by its number of quantifiers.

Terms versus circuits. A *term* is a syntax tree, so repeated occurrences of a subterm are distinct nodes. A *relational circuit* is a finite directed acyclic graph (DAG): each gate is an input, a constant, or one of the same CoR operations, with ordered input ports, and a designated output gate. A gate may feed multiple later gates, including both ports of one composition. Its value $[\![g]\!]_{\mathcal{M}}$ is computed once from its predecessors; $[\![C]\!]_{\mathcal{M}}$ is the value of the output gate.

For example, the term $(R;R);(R;R)$ has seven nodes and three composition occurrences. The circuit

$$g_0 \coloneqq R, \qquad g_1 \coloneqq g_0; g_0, \qquad g_2 \coloneqq g_1; g_1$$

has just three distinct gates, two of which are compositions (Figure 5).

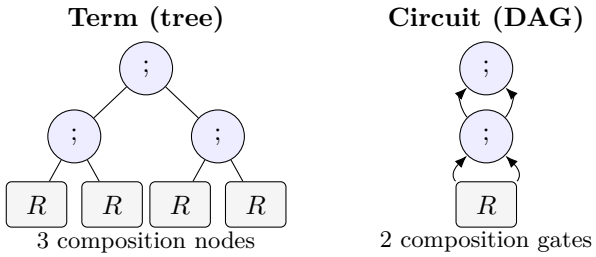


Figure 5: The term $(R;R);(R;R)$ repeats the same subterm, whereas the circuit reuses one gate as both inputs to the next composition. Distinct gates, not wires or uses, are counted.

More generally, g_d defined by $g_{i+1} \coloneqq g_i; g_i$ computes the pairs joined by an R-walk of length 2^d. Fully unfolding this computation gives a term with $2^d - 1$ compositions, while the circuit has only d composition gates. For another example, a circuit can compute $h \coloneqq R; S$ once and use it in both branches of $(P \cap h) \cup (Q \cap h^\smile)$, still with only one composition gate. Thus sharing can save exponentially many compositions; our lower bounds hold *despite* allowing it.

Equations. A CoR *equation* is a statement $T = S$, satisfied by a structure precisely when

$$\mathcal{M} \models T = S \quad \iff \quad [\![T]\!]_{\mathcal{M}} = [\![S]\!]_{\mathcal{M}}.$$

For example, $\mathbf{id} \cup R = \mathbf{1}$ expresses the FO^2 sentence $\forall x \, \forall y \, (x = y \vee R(x, y))$. A circuit computing an equation may have two designated output gates (for T and S), with shared intermediate gates counted only once.

2.3 Resource measures and computational conventions

Syntax size and gate counts. Formula size $|\varphi|$ and term size $|T|$ count nodes in their *syntax trees*; all connectives have arity at most two. Negation and quantification are unary, and displayed iterated connectives are abbreviations to be expanded. Each relation-atom occurrence counts as one node even for a growing signature (so $|G_k| = 8k$); the length of the written relation name is not charged. This is different from measuring the encoded bit length of a formula. Write $\#_{\mathrm{rel}}(\varphi)$ for the number of relation-atom occurrences (including $R(x, x)$, but excluding equality atoms), and $\#_{\exists\forall}(\varphi)$ for its quantifier occurrences.

For a term T, write $\#_{\mathrm{rel}}(T)$ for occurrences of input relation symbols and $\#_{;}(T)$ for occurrences of composition. For a circuit C, $|C|$ counts its distinct gates, including input and constant gates, and $g_{;}(C)$ counts distinct composition gates. All other operations are uncharged by $g_{;}(C)$, though they do count toward $|C|$. A shared gate counts once, irrespective of its fan-out. We will also use the elementary tree inequalities

$$|T| \geq 2\#_{;}(T) + 1, \qquad |T| \geq 2\#_{\mathrm{rel}}(T) - 1.$$

Equivalence and allowed inputs. For a formula φ and a relational term or circuit C over the same signature, write

$$C \equiv_{\mathrm{fin}} \varphi \quad \iff \quad [\![C]\!]_{\mathcal{M}} = [\![\varphi]\!]_{\mathcal{M}} \quad \text{for every finite } \sigma\text{-structure } \mathcal{M}.$$

The same notation applies to terms and to comparisons of formulas. Unsubscripted equivalences in our explicit translations mean agreement over all nonempty structures. A *size-specific* circuit C_n may depend on $n = |D|$, but not on the input interpretation; it must compute the intended query on *every* n-element input. A relational circuit has only the input symbols and the three constants $\mathbf{0}, \mathbf{1}, \mathbf{id}$; no built-in order, named vertices, extra vertex predicates, or arbitrary vertex-dependent constants are supplied for free. The more permissive typed matrix circuit model of Section 7.2 is defined separately.

Randomized targets. A randomized circuit is an input-independent distribution $\mathcal{D}_n$ over deterministic circuits, possibly of different sizes, for the given domain size n. Pointwise error at most ε for a query q means that, for every n-element input $\mathcal{M}$ and every $(a, b) \in D^2$,

$$\Pr_{C \sim \mathcal{D}_n} [[\![C]\!]_{\mathcal{M}}(a, b) = [\![q]\!]_{\mathcal{M}}(a, b)] \geq 1 - \varepsilon.$$

The entire output relation need not be correct simultaneously. Randomized resource bounds concern expectations over $\mathcal{D}_n$; the circuit distribution is fixed independently of the input.

Common notation. We use $[k] \coloneqq \{1, \dots, k\}$ and $2^{[k]}$ for its power set; $A^c \coloneqq [k] \setminus A$ when $A \subseteq [k]$. Empty intersections and unions of relations mean $\mathbf{1}$ and $\mathbf{0}$, respectively; the analogous Boolean operations mean true and false. Truth values are $\mathbb{B} \coloneqq \{0, 1\}$, and $[\mathcal{P}]$ is the indicator of a statement $\mathcal{P}$. The special palette, hub, and fold-prediction notation are introduced when needed in Section 3.

3 Folds, predictions, and the test–cost theorem

This section proves the machinery of the lower bounds in the order of the overview. We construct the reference structure (§3.1) and define color-determined relations, folds, and predictions (§3.2). We then show that a relational circuit computes the predicted values on every folded structure whose fold none of its compositions notices (§3.3), and that each composition notices at most two folds (§3.4). With the tests of §4.1, this already proves the composition bounds of Theorem 1.1. Section 3.5 states the test–cost theorem, a counting form of the same argument that also covers randomized circuits and the other targets of §1.4; Section 7 supplies its hypotheses for scalar-valued computations.

3.1 The reference structure

Fix a palette Λ of $N \geq 2$ colors and a vertex set V of size n. A coloring $\chi : \binom{V}{2} \to \Lambda$ assigns a color to every unordered pair of distinct vertices. We write $\chi(u, v)$ for $\chi(\{u, v\})$.

Lemma 3.1 (Two witnesses)**.** *For every integer*

$$n \geq n_0(N) \coloneqq \left\lceil 12N^2 \ln(2N) \right\rceil + 3,$$

there is a coloring such that, for all distinct $u, v \in V$ and all $\mu, \nu \in \Lambda$, at least two distinct vertices $z \in V \setminus \{u, v\}$ satisfy $\chi(u, z) = \mu$ and $\chi(z, v) = \nu$.

Proof. Color the edges independently and uniformly. For fixed distinct u, v, w and colors μ, ν, the probability that no suitable witness occurs outside $\{u, v, w\}$ is at most $\exp(-(n-3)/N^2)$: the candidate witnesses use disjoint pairs of edges. A union bound gives failure probability at most

$$n^3 N^2 \exp(-(n-3)/N^2).$$

At $n = n_0(N)$, the inequality $n_0(N) \leq 13N^3$ bounds this expression by $2197/(4096N) < 1$. It decreases for $n > 3N^2$, proving the assertion at every larger size. Hence some coloring has, for all distinct u, v, w and all colors μ, ν, a witness outside $\{u, v, w\}$; applying this with w equal to one witness for u, v yields a second. □

Fix such a coloring, and choose a vertex $\hbar \in V$, called the hub. The two witnesses of Lemma 3.1 play different roles. One witness outside the endpoints already makes every computed relation a union of color classes, possibly together with the diagonal (Lemma 3.2). The second ensures that two vertices other than $\hbar$ have, for every pair of colors, a witness that also avoids $\hbar$, so that recoloring the edges at $\hbar$ does not affect pairs away from it (§3.4). Lemma 3.1 also implies that every vertex sees every color, and that every vertex other than $\hbar$ sees every color on edges avoiding $\hbar$ (take $v = \hbar$). For each color μ, fix a vertex v_μ with $\chi(\hbar, v_\mu) = \mu$. No vertex, color, or color neighborhood is supplied as an additional input predicate.

Jipsen, Maddux, and Tuza [JMT95] gave the random and affine-plane colorings underlying our reference structures. Although they formulated a one-witness condition, their constructions also yield the two-witness property used here. The random-coloring argument above establishes it at every sufficiently large domain size; their affine-plane coloring has it for planes of order at least four. Appendix A verifies the latter and gives an explicit construction on fewer than $16N^2$ vertices.

3.2 Color-determined relations, folds, and predictions

For $\mu \in \Lambda$, let $e_\mu \coloneqq \{(u,v) \in V^2 : u \neq v,\ \chi(u,v) = \mu\}$ be the set of edges of color μ, in both orientations. We extend χ to loops by giving every loop the extra label $\mathbf{id}$, which is not a color. A relation $X \subseteq V^2$ is *color-determined* if $X(u,v)$ depends only on $\chi(u,v)$. For such a relation, write $X[\mu] \in \mathbb{B}$ for its common value at pairs (u,v) with $\chi(u,v) = \mu$, where $\mu \in \Lambda \cup \{\mathbf{id}\}$. These $N+1$ values form a table, and

$$S_X \coloneqq \{\mu \in \Lambda : X[\mu] = 1\}$$

is the *off-diagonal support* of X. Thus X is the union of the classes e_μ with $\mu \in S_X$, together with the diagonal if $X[\mathbf{id}] = 1$. A reference structure interprets every input relation by a color-determined relation; we write $\mathcal{M}_0$ for it.

Lemma 3.2 (Closure of color-determined relations)**.** *The constants $\mathbf{0}, \mathbf{1}, \mathbf{id}$ are color-determined, and color-determined relations are closed under Boolean operations, converse, and composition. If X and Y are color-determined, then for every color $\mu \in \Lambda$,*

$$\begin{aligned} (X;Y)[\mu] &= (X[\mathbf{id}] \wedge Y[\mu]) \vee (X[\mu] \wedge Y[\mathbf{id}]) \vee (S_X \neq \varnothing \wedge S_Y \neq \varnothing), \\ (X;Y)[\mathbf{id}] &= (X[\mathbf{id}] \wedge Y[\mathbf{id}]) \vee (S_X \cap S_Y \neq \varnothing), \end{aligned} \tag{CT}$$

where a condition stands for its truth value.

Proof. Boolean operations act entry by entry, and converse preserves this property because χ is symmetric. For composition, consider a pair (u,v) and a potential witness z. An endpoint witness, $z \in \{u,v\}$, contributes the disjuncts of (CT) that involve $X[\mathbf{id}]$ or $Y[\mathbf{id}]$. An outside witness, $z \notin \{u,v\}$, contributes $X[\chi(u,z)] \wedge Y[\chi(z,v)]$. If $u \neq v$, then by Lemma 3.1 the color pairs $(\chi(u,z), \chi(z,v))$ of outside witnesses range over all of $\Lambda \times \Lambda$, so an outside witness exists exactly when S_X and S_Y are both nonempty. If $u = v$, both edges of an outside witness are the edge $\{u,z\}$, and every color occurs on an edge at u, so an outside witness exists exactly when $S_X \cap S_Y \neq \varnothing$. □

In particular, every value of a relational circuit on $\mathcal{M}_0$ is color-determined, and

$$e_\mu ; e_\nu = \begin{cases} \mathbf{1}, & \mu = \nu, \\ \overline{\mathbf{id}}, & \mu \neq \nu. \end{cases}$$

Thus the color-determined relations give a concrete representation of the symmetric relation algebra $E_{N+1}(1,2,3)$ studied by Jipsen, Maddux, and Tuza [JMT95]. Its atoms are $\mathbf{id}$ and the e_μ, and $\overline{\mathbf{id}} = \bigcup_{\mu \in \Lambda} e_\mu$ is the diversity relation. This identification follows from the displayed multiplication table, not from automorphisms of the reference graph.

Folds and predictions. From now on, take the palette $\Lambda \coloneqq 2^{[k]}$ for some $k \geq 1$, so that $N = 2^k$. The complement of a color μ is the color $\mu^c = [k] \setminus \mu$, and no color equals its complement. The fold of a color λ recolors every edge of color λ at the hub to λ^c and leaves all other edges unchanged: for $u \neq v$,

$$\chi_\lambda(u,v) \coloneqq \begin{cases} \lambda^c, & \hbar \in \{u,v\} \text{ and } \chi(u,v) = \lambda, \\ \chi(u,v), & \text{otherwise,} \end{cases}$$

and every loop keeps the label **id**. Only two properties of the fold matter below: it removes exactly the color λ from the hub, and it changes no edge away from the hub. Any target color other than λ would serve as well. For a color-determined relation X, its prediction $X^{(\lambda)}$ reads the same values on χ_λ:

$$X^{(\lambda)}(u,v) \coloneqq X[\chi_\lambda(u,v)] \qquad (u,v \in V).$$

Thus $X^{(\lambda)}$ agrees with X except on the recolored edges, where it takes the value $X[\lambda^c]$. Replacing every input relation by its prediction defines the folded structure $\mathcal{M}_\lambda$ on the same domain V. Predictions are defined only for color-determined relations, in particular for all values computed on $\mathcal{M}_0$. We do not assume that relations computed on $\mathcal{M}_\lambda$ are color-determined; in general, they are not.

3.3 Noticing, and unnoticed folds

Let λ be a color and $\mathbf{U} = (U_1, \ldots, U_r)$ a tuple of color-determined relations. An r-ary operation op notices the fold of λ at $\mathbf{U}$ if

$$\mathrm{op}(U_1^{(\lambda)}, \ldots, U_r^{(\lambda)}) \neq \mathrm{op}(\mathbf{U})^{(\lambda)}.$$

Both sides are determined by $\mathbf{U}$ and λ: noticing is a property of the operands on $\mathcal{M}_0$, not of any computation on $\mathcal{M}_\lambda$.

Proposition 3.3 (Commutation with prediction)**.** *For any color λ and color-determined relations X, Y, prediction commutes with Boolean operations and converse:*

$$(X \cup Y)^{(\lambda)} = X^{(\lambda)} \cup Y^{(\lambda)}, \quad (X \cap Y)^{(\lambda)} = X^{(\lambda)} \cap Y^{(\lambda)},$$
$$(\overline{X})^{(\lambda)} = \overline{X^{(\lambda)}}, \qquad (X^\smile)^{(\lambda)} = (X^{(\lambda)})^\smile.$$

The constants $\mathbf{0}, \mathbf{1}, \mathbf{id}$ are also fixed by prediction.

Proof. Boolean operations act pair by pair, so both sides apply the same operation to the entries at $\chi_\lambda(u,v)$. Converse commutes with prediction because $\chi_\lambda(u,v) = \chi_\lambda(v,u)$. The constants are fixed because a fold preserves the diagonal. □

Thus only composition can notice a fold. A composition gate of a relational circuit notices the fold of λ if the composition of its operand values on $\mathcal{M}_0$ notices it.

Lemma 3.4 (Unnoticed folds)**.** *Let C be a relational circuit and λ a color. If no composition gate of C notices the fold of λ, then on $\mathcal{M}_\lambda$ every gate of C computes the prediction of its value on $\mathcal{M}_0$. In particular, the outputs of C on $\mathcal{M}_0$ and on $\mathcal{M}_\lambda$ agree at every pair that the fold does not recolor.*

Proof. Evaluate C on $\mathcal{M}_0$. By Lemma 3.2, every gate value is color-determined, so its prediction is defined. We show by induction along a topological order of the gates that every gate computes on $\mathcal{M}_\lambda$ the prediction of its value on $\mathcal{M}_0$. For input relations this is the definition of $\mathcal{M}_\lambda$, and constants are their own predictions by Proposition 3.3. For Boolean and converse gates, the claim follows from the induction hypothesis and Proposition 3.3. If the operands of a composition gate have values X and Y on $\mathcal{M}_0$, then by induction they have values $X^{(\lambda)}$ and $Y^{(\lambda)}$ on $\mathcal{M}_\lambda$. The gate therefore computes $X^{(\lambda)};Y^{(\lambda)}$ on $\mathcal{M}_\lambda$, which equals $(X;Y)^{(\lambda)}$ because the gate does not notice the fold. Finally, every value U on $\mathcal{M}_0$ agrees with $U^{(\lambda)}$ at every pair that the fold does not recolor. □

3.4 Each composition notices at most two folds

Fix color-determined relations X, Y and a color λ, and write $J \coloneqq \Lambda \setminus \{\lambda\}$ for the set of colors at $\hbar$ after the fold of λ. Endpoint witnesses contribute the same value to $X^{(\lambda)};Y^{(\lambda)}$ and to $(X;Y)^{(\lambda)}$. At a pair (u,v) with $u \neq v$ and $\mu \coloneqq \chi_\lambda(u,v)$, they contribute $(X[\mathbf{id}] \wedge Y[\mu]) \vee (X[\mu] \wedge Y[\mathbf{id}])$ to $X^{(\lambda)};Y^{(\lambda)}$, because $X^{(\lambda)}$ and $Y^{(\lambda)}$ take the values $X[\mathbf{id}]$ and $Y[\mathbf{id}]$ on the diagonal and $X[\mu]$ and $Y[\mu]$ at (u,v), and the same value to $(X;Y)^{(\lambda)}(u,v) = (X;Y)[\mu]$ by (CT); at a diagonal pair, they contribute $X[\mathbf{id}] \wedge Y[\mathbf{id}]$ to both. The following table compares the outside witnesses. Its left column states when $X^{(\lambda)};Y^{(\lambda)}$ has an outside witness at the pair, and its right column states the last disjunct of (CT), which $(X;Y)^{(\lambda)}$ reads at the pair; here u and v range over vertices other than $\hbar$, and $S \neq \varnothing \neq S'$ means that both sets are nonempty.

Output pair	Outside witness in $X^{(\lambda)};Y^{(\lambda)}$	Last disjunct of (CT)
(u,v), $u \neq v$	$S_X \neq \varnothing \neq S_Y$	$S_X \neq \varnothing \neq S_Y$
(u,u)	$S_X \cap S_Y \neq \varnothing$	$S_X \cap S_Y \neq \varnothing$
$(\hbar,v)$	$S_X \cap J \neq \varnothing \neq S_Y$	$S_X \neq \varnothing \neq S_Y$
$(u,\hbar)$	$S_X \neq \varnothing \neq S_Y \cap J$	$S_X \neq \varnothing \neq S_Y$
$(\hbar,\hbar)$	$S_X \cap S_Y \cap J \neq \varnothing$	$S_X \cap S_Y \neq \varnothing$

For the left column, consider the outside witnesses z of a pair. For distinct $u, v \neq \hbar$, Lemma 3.1 gives, for every pair of colors, an outside witness other than $\hbar$, whose two edges keep their colors; the witness $\hbar$ contributes one more color pair, which is already realized. For (u,u) with $u \neq \hbar$, every color occurs on an edge at u that avoids $\hbar$. At a pair $(\hbar,v)$ of the *hub row*, the left edge $\{\hbar,z\}$ of every outside witness lies at the hub, so its color becomes $\chi_\lambda(\hbar,z) \in J$, while the right edge $\{z,v\}$ keeps its color; by Lemma 3.1 the realized color pairs are exactly $J \times \Lambda$. The pairs $(u,\hbar)$ of the *hub column* are symmetric, with $\Lambda \times J$. At $(\hbar,\hbar)$ both edges of an outside witness are the edge $\{\hbar,z\}$, whose color $\chi_\lambda(\hbar,z)$ ranges exactly over J.

Lemma 3.5 (When a composition notices a fold)**.** *Let X, Y be color-determined. The composition $X;Y$ notices the fold of λ only if*

$$S_X = \{\lambda\}, \qquad S_Y = \{\lambda\}, \qquad \text{or} \qquad S_X \cap S_Y = \{\lambda\}.$$

Hence it notices at most two folds, at most one if S_X or S_Y is $\varnothing$ or Λ, and none if both are.

Proof. By the table, the two sides can differ only in the hub row, in the hub column, or at $(\hbar,\hbar)$. In the hub row they can differ only if $S_X \neq \varnothing$ but $S_X \cap J = \varnothing$, that is, $S_X = \{\lambda\}$; in the hub

column only if $S_Y = \{\lambda\}$; and at $(\hbar, \hbar)$ only if $S_X \cap S_Y = \{\lambda\}$. Each condition determines λ. If $S_X \cap S_Y \neq \varnothing$, each of the first two conditions implies the third, so at most one fold is noticed; otherwise the third condition never holds, and at most two are. If S_Y is $\varnothing$ or Λ, then $S_Y \neq \{\lambda\}$ because $N \geq 2$, and $S_X \cap S_Y$ is $\varnothing$ or S_X, so only the condition $S_X = \{\lambda\}$ remains; the case of S_X is symmetric. If both are $\varnothing$ or Λ, none of the conditions can hold. □

The bound two is attained. The composition $e_\mu; e_{\mu^c}$ has $S_X = \{\mu\}$ and $S_Y = \{\mu^c\}$, and notices the folds of both μ and μ^c. On the reference structure of §4.1, the relation e_μ is the relation H_μ of the DNF. Proposition B.1 computes G_k on the reference structure and all its folded structures with 2^{k-1} such compositions. By Lemma 3.5, each composition gate notices at most two folds. What a gate notices depends only on its operand values on $\mathcal{M}_0$, so a shared gate counts once, regardless of its fan-out. Hence at most $2g_;(C)$ folds are noticed by some gate of C, and Lemma 3.4 applies to all other folds. Nothing here depends on the query or on the interpretation of the input relations. If a query has different answers on $\mathcal{M}_0$ and on $\mathcal{M}_\lambda$ at a pair that the fold of λ does not recolor, for every color λ, then every relational circuit C defining it satisfies $2g_;(C) \geq 2^k$. Section 4 shows that G_k is such a query.

3.5 Tests and the test–cost theorem

For randomized circuits and for the other targets of §1.4, we use a counting form of this argument that refers only to a few abstract properties. For a color λ, the *test* for λ consists of the structures $\mathcal{M}_0$ and $\mathcal{M}_\lambda$ and the *output observation* $\rho_\lambda(U) \coloneqq U(\hbar, v_{\lambda^c})$, the entry of an output value U at the pair $(\hbar, v_{\lambda^c})$ of §3.1. Since $\lambda^c \neq \lambda$, the fold of λ does not recolor this pair, so

$$\rho_\lambda(U^{(\lambda)}) = \rho_\lambda(U) \tag{O}$$

for every reference output value U, as defined in (i) below.

The counting argument uses only three properties of this construction. We call data satisfying them a *test system*:

(i) Each type of value has a collection of *reference values*, which contains the input values on $\mathcal{M}_0$ and is closed under the permitted operations, and every reference value U has a prediction $U^{(\lambda)}$ for each color λ.

(ii) Every input value on $\mathcal{M}_\lambda$ is the prediction of the same input value on $\mathcal{M}_0$.

(iii) Every test satisfies (O), and the required answers at its observation on $\mathcal{M}_0$ and on $\mathcal{M}_\lambda$ differ.

For relational circuits, the reference values are the color-determined relations, and Lemma 3.2 supplies (i). The typed scalar computations in §7 will satisfy the same conditions; predictions carry the type of their argument, which we suppress in the notation. For a gate g of a deterministic circuit C, let op_g be its operation and $\mathbf{U}_g$ the tuple of its operand values on $\mathcal{M}_0$. As in §3.3, g notices the fold of λ if $\mathrm{op}_g(\mathbf{U}_g^{(\lambda)}) \neq \mathrm{op}_g(\mathbf{U}_g)^{(\lambda)}$, where predictions act componentwise on tuples. Define

$$D_g \coloneqq \{\lambda \in \Lambda : g \text{ notices the fold of } \lambda\},$$
$$\mathrm{cost}(C) \coloneqq \sum_{g \text{ distinct}} |D_g|.$$

For input nodes put $D_g \coloneqq \varnothing$; constant gates are nullary operations. The definition of D_g compares the entire output of the gate, and it uses predicted operands computed from $\mathcal{M}_0$, not operands recomputed on $\mathcal{M}_\lambda$.

We call $\operatorname{cost}(C)$ the *cost* of C. It counts the pairs of a distinct gate and a fold that the gate notices. This integer depends on the reference structure and the tests, which remain fixed, but is defined even for an incorrect circuit.

Theorem 3.6 (Test–cost theorem)**.** *Fix a test system. A deterministic circuit C correct at both observations of every test satisfies*

$$2^k \le \operatorname{cost}(C).$$

If C_ω is chosen from an input-independent distribution of circuits, and each observation in each test is incorrect with probability at most $\varepsilon < 1/2$, then

$$(1 - 2\varepsilon)2^k \le \mathbb{E}_\omega \operatorname{cost}(C_\omega).$$

Proof. Fix a deterministic circuit, not assumed correct, and put $D_C \coloneqq \bigcup_g D_g$. For $\lambda \notin D_C$, no gate notices the fold of λ, so the induction in the proof of Lemma 3.4, with (ii) for the input values, gives

$$[\![C]\!]_{\mathcal{M}_\lambda} = ([\![C]\!]_{\mathcal{M}_0})^{(\lambda)}.$$

The equality concerns the whole output. By the observation condition (O), the two observations are equal, whereas their correct values differ. Hence at least one is wrong. If $\operatorname{err}_{\lambda,0}(C), \operatorname{err}_{\lambda,1}(C)$ are the corresponding zero-one error indicators, then

$$\operatorname{cost}(C) + \sum_{\lambda \in \Lambda} \big(\operatorname{err}_{\lambda,0}(C) + \operatorname{err}_{\lambda,1}(C)\big) \ge 2^k,$$

because $|D_C| \le \operatorname{cost}(C)$. Zero error proves the first assertion. Take expectations using the same random circuit on the two structures of each test. Each error indicator has expectation at most ε, which proves the second assertion. No independence among the tests or their errors is required. □

This is the counting argument for all subsequent lower bounds on the reference structure. Each of them bounds $\operatorname{cost}(C)$ from above by a resource count of the target, such as $2g_;(C)$ in Lemma 3.7 below, and therefore also holds in two further forms, which we do not restate. First, Lemma 3.1 gives the reference structure on every domain of size $n \ge n_0(2^k)$, and the folds keep the domain, so each bound holds for targets that need to be correct only on n-element inputs and may be chosen for n. Second, by the randomized assertion of Theorem 3.6, each bound for targets that compute a query also holds for an input-independent distribution of targets with pointwise error at most $\varepsilon < 1/2$, after the resource count is replaced by its expectation and the bound is multiplied by $1 - 2\varepsilon$; for scalar outputs, an error is a failure to return the exact required value.

For ordinary CoR, call a composition gate whose operand values on $\mathcal{M}_0$ are X, Y *one-sided* if S_X or S_Y is $\varnothing$ or Λ. Let $m_1(C)$ count these gates and $m_2(C)$ the other composition gates, counting shared gates once.

Lemma 3.7 (Cost of compositions)**.** *Every relational circuit C satisfies*

$$\operatorname{cost}(C) \le m_1(C) + 2m_2(C) \le 2g_;(C).$$

Proof. By Lemma 3.5, a one-sided composition notices at most one fold and any other composition at most two. Other gates notice nothing. Summing over distinct gates proves the inequalities. □

The projections $X;\mathbf{1}$ and $\mathbf{1};X$ are one-sided. If a syntactic count is preferred, classify only these projections as one-sided and put every remaining composition in m_2; Lemma 3.7 still holds.

4 Lower bounds for relational circuits

The construction of §3 is independent of the query. We now interpret the input relations of G_k on it; the read edge of each test identifies the witness color that the fold removes.

4.1 The tests for G_k

On a color μ, interpret P_i as 1 precisely when $i \in \mu$, and Q_i as 1 precisely when $i \notin \mu$. Put both values zero on every loop. More explicitly,

$$P_i^{\mathcal{M}_0} := \bigcup_{\mu\in\Lambda:\, i\in\mu} e_\mu, \qquad Q_i^{\mathcal{M}_0} := \bigcup_{\mu\in\Lambda:\, i\notin\mu} e_\mu.$$

For each color λ, use the read edge $(\hbar, v_{\lambda^c})$, of color λ^c (§3.1), and the fold of λ (§3.2). Since $\lambda \neq \lambda^c$, the fold does not recolor this read edge.

Lemma 4.1 (The tests for G_k)**.** *For every color λ,*

$$[\![G_k]\!]_{\mathcal{M}_0}(\hbar, v_{\lambda^c}) = 1, \qquad [\![G_k]\!]_{\mathcal{M}_\lambda}(\hbar, v_{\lambda^c}) = 0.$$

Moreover, the sentence $G_k^\forall = \forall x\,\forall y\,(x = y \vee G_k(x,y))$ of §1.1 is true in $\mathcal{M}_0$ and false in every $\mathcal{M}_\lambda$. Thus G_k and $G_k^\forall$ each supply 2^k tests.

Proof. For non-loop edges with colors μ, ν, the body of G_k holds exactly when $\nu = \mu^c$. A loop, whose P_i, Q_i values are all zero, cannot be a witness edge. Hence, on $\mathcal{M}_0$ and on every $\mathcal{M}_\lambda$, G_k holds at a pair (u, v) of distinct vertices exactly when u has an edge whose color is the complement of the color of (u, v). Against the read color λ^c, the sole accepted witness color is λ. That color occurs at $\hbar$ in $\mathcal{M}_0$ and does not occur there after the fold. In $\mathcal{M}_0$, every vertex has edges of every color, so G_k holds at every pair of distinct vertices, and $x = y$ handles the diagonal; in $\mathcal{M}_\lambda$, G_k fails at the pair $(\hbar, v_{\lambda^c})$ of distinct vertices. □

4.2 Consequences

Theorem 1.1 (Exponential lower bounds)**.** *For every $k \geq 1$, every* CoR *circuit $C \equiv_{\text{fin}} G_k$ has at least 2^{k-1} composition gates, and every* CoR *term $T \equiv_{\text{fin}} G_k$ has at least 2^{k-1} compositions.*

Proof. Let $C \equiv_{\text{fin}} G_k$, and take the reference structure at any size $n \geq n_0(2^k)$. For each color λ, the answers of G_k at $(\hbar, v_{\lambda^c})$ on $\mathcal{M}_0$ and on $\mathcal{M}_\lambda$ differ by Lemma 4.1, and the fold of λ does not recolor this pair. By Lemma 3.4, some composition gate of C notices the fold of λ. By Lemma 3.5, each composition gate notices at most two folds, so $2g_;(C) \geq 2^k$. A term is a circuit without sharing, so the same bound holds for its number of compositions. □

Together with the expansion of G_k in §1.1, these lower bounds show that the minimum circuit size for G_k is $\Theta(2^k)$; Theorem 6.3 in §6.2 determines the minimum term size.

The lower bound does not rest on the difficulty of evaluating G_k. As its proof shows, it holds even for circuits that are correct only on the reference structure and its folded structures, on which every non-loop edge has a color from the palette. A read edge there accepts exactly the witness edges of the complementary color, so a trie of the colors present in each row evaluates G_k on all pairs of these structures in time $O(kn^2)$, linear in the size of the input.

Equations. By the semantics of CoR equations in §2.2, the same tests also bound the equational form of Tarski's correspondence.

Corollary 4.2 (CoR equations)**.** *If a* CoR *equation $T = S$ holds on exactly those finite structures that satisfy $G_k^\forall$, then every relational circuit with gates computing T and S has at least $2^{k-1} - 1$ composition gates.*

Proof. Let C have gates computing T and S, and add gates for

$$\mathrm{Eq} \coloneqq \overline{\mathbf{1}; ((T \cap \overline{S}) \cup (\overline{T} \cap S)); \mathbf{1}},$$

which is the full relation when $T = S$ and empty otherwise. This adds two compositions, both one-sided. By Lemma 4.1, $G_k^\forall$ is true in $\mathcal{M}_0$ and false in every $\mathcal{M}_\lambda$, so Eq is full on $\mathcal{M}_0$ and empty on every $\mathcal{M}_\lambda$. Both constants are their own predictions, so these are 2^k tests, and Theorem 3.6 with Lemma 3.7 gives $2^k \le 2g_;(C) + 2$. □

The DNF of §1.1 gives the equivalent equation $\mathbf{id} \cup \bigcup_{A \subseteq [k]} [H_A \cap (H_{A^c}; \mathbf{1})] = \mathbf{1}$ with 2^k compositions, so the bound is tight up to a factor of two.

Fixed sizes and bounded error. The reference structure and its folded structures share one domain, which may have any size $n \ge n_0(2^k)$, and the randomized assertion of Theorem 3.6 handles bounded error.

Corollary 1.8 (Fixed sizes and bounded error)**.** *For some constant $c > 0$, for every $k \ge 1$, $n \ge ck4^k$, and $0 \le \varepsilon < 1/2$, every input-independent distribution of size-specific* CoR *circuits whose output at each pair (u, v) is correct with probability at least $1 - \varepsilon$ on every n-element input satisfies*

$$\mathbb{E}[g_;(C)] \ge (1 - 2\varepsilon)2^{k-1}.$$

Proof. At the prescribed size n, Lemma 3.1 gives the reference structure, and Lemma 4.1 gives all 2^k tests on that domain. Apply the expected-cost assertion of Theorem 3.6 and $\mathrm{cost}(C) \le 2g_;(C)$. Setting $\varepsilon = 0$ includes every exact size-specific circuit. □

Circuits without converse. Without converse, a directed version of the reference structure lets each composition notice at most one fold, and the bound becomes exact.

Corollary 4.3 (Without converse)**.** *For every $k \ge 1$, every relational circuit without converse that defines G_k on finite structures has at least 2^k composition gates. This bound is attained by the DNF.*

Appendix C proves the corollary with a directed reference coloring. For full CoR, Proposition B.1 gives a circuit with 2^{k-1} compositions that is correct on the reference structure and all its folded structures. Thus these tests attain the coefficient two of Lemma 3.5 and cannot give more than 2^{k-1}. The optimum for G_k on arbitrary inputs remains between 2^{k-1} and 2^k.

4.3 Domain size

On arbitrary inputs, the *profile* $\pi(x,z) \in \{0,1\}^{2k}$ of a pair lists the values of $P_1,\dots,P_k,Q_1,\dots,Q_k$ at (x,z); on the reference structure, the profile of an edge is determined by its color. By the definition of G_k, the read edge (x,y) accepts the witness edge (x,z) exactly when, for every i, the profile $\pi(x,y)$ has P_i and $\pi(x,z)$ has Q_i, or $\pi(x,y)$ has Q_i and $\pi(x,z)$ has P_i. Hence $G_k(x,y)$ depends only on $\pi(x,y)$ and the set $\Pi_x \coloneqq \{\pi(x,z) : z \in V\}$ of profiles in row x, which has at most $|V|$ elements.

Proposition 4.4 (Small domains)**.** *For all $k,n \geq 1$ there is a converse-free relational circuit with $4kn$ composition gates, all of them row projections $X;\mathbf{1}$, and $O(k^2n^2)$ other gates, that computes G_k on every structure with at most n elements.*

Proof. A relation is *row-constant* if its value at (x,y) does not depend on y. By (Proj), each row projection $S;\mathbf{1}$ is row-constant and holds in row x exactly when some pair (x,z) lies in S; Boolean combinations of row-constant relations are row-constant. For $1 \leq b \leq 2k$, let E_b be the input relation giving bit b of the profile, and put $E_b^1 \coloneqq E_b$ and $E_b^0 \coloneqq \overline{E_b}$.

In a structure with at most n elements, call a word $w \in \{0,1\}^\ell$ *live* in row x if some profile in Π_x begins with w. Live words of one length begin distinct profiles, so there are at most n of each length, and the empty word is live because $\pi(x,x) \in \Pi_x$. The circuit stores the live words of length ℓ in n slots: row-constant relations $L_{\ell,r}$ and $W_{\ell,r,b}$, for $r \leq n$ and $b \leq \ell$, state that slot r is occupied in row x and that its word has bit b. Initially $L_{0,1} \coloneqq \mathbf{1}$ and $L_{0,r} \coloneqq \mathbf{0}$ for $2 \leq r \leq n$; the only stored word is empty, so there are no word bits. For each slot r and each $\sigma \in \{0,1\}$, the relation

$$S_{\ell,r,\sigma} \coloneqq L_{\ell,r} \cap \bigcap_{b=1}^{\ell} \left((E_b \cap W_{\ell,r,b}) \cup (\overline{E_b} \cap \overline{W_{\ell,r,b}})\right) \cap E_{\ell+1}^{\sigma}$$

holds at (x,z) exactly when slot r is occupied in row x and $\pi(x,z)$ begins with its word followed by σ. The row projection $S_{\ell,r,\sigma};\mathbf{1}$ therefore tells, in every row at once, whether this extension is live. The at most n live extensions are placed into the slots of length $\ell+1$ in the fixed order of their pairs (r,σ); no order on the input vertices is used. Index the $2n$ candidate pairs by (r_i,σ_i), for $1 \leq i \leq 2n$, and put $f_i \coloneqq S_{\ell,r_i,\sigma_i};\mathbf{1}$. Using only Boolean gates, compute, for $0 \leq i \leq 2n$ and $0 \leq j \leq n$, the row-constant relation $C_{i,j}$ that holds in a row exactly when j of $f_1,\dots,f_i$ hold:

$$C_{0,0} \coloneqq \mathbf{1}, \qquad C_{0,j} \coloneqq \mathbf{0}\ (j>0), \qquad C_{i,j} \coloneqq (\overline{f_i} \cap C_{i-1,j}) \cup (f_i \cap C_{i-1,j-1}),$$

where $1 \leq i \leq 2n$, $0 \leq j \leq n$, and $C_{i,-1} \coloneqq \mathbf{0}$. For $1 \leq j \leq n$, put

$$A_{i,j} \coloneqq f_i \cap C_{i-1,j-1} \qquad (1 \leq i \leq 2n).$$

In each row, $A_{i,j}$ holds exactly if candidate i is the jth live candidate. In particular, at most one

$A_{i,j}$ holds for a given slot j. Thus occupancy and every bit of the new slots are computed by

$$L_{\ell+1,j} \coloneqq \bigcup_{i=1}^{2n} A_{i,j},$$

$$W_{\ell+1,j,b} \coloneqq \bigcup_{i=1}^{2n} (A_{i,j} \cap W_{\ell,r_i,b}) \quad (1 \le b \le \ell),$$

$$W_{\ell+1,j,\ell+1} \coloneqq \bigcup_{\substack{1\le i\le 2n\\ \sigma_i=1}} A_{i,j}.$$

Distinct live prefixes have distinct extensions, and every live word of length $\ell+1$ extends a live prefix. Thus these formulas preserve the slot invariant. They use only Boolean operations on row-constant relations, so no further compositions are needed. At each length, the $2n$ projections producing the f_i are the only composition gates. The candidate tests use $O((\ell+1)n)$ Boolean gates, the prefix counts and ranks use $O(n^2)$, and updating all slots and their $\ell+1$ bits uses $O((\ell+1)n^2)$. Summing over the $2k$ lengths, together with the final output gates below, yields $O(k^2n^2)$ non-composition gates. After length $2k$ the occupied slots hold exactly the profiles in Π_x, and

$$G_k = \bigcup_{r=1}^{n} \Big(L_{2k,r} \cap \bigcap_{i=1}^{k} ((P_i \cap W_{2k,r,k+i}) \cup (Q_i \cap W_{2k,r,i})) \Big),$$

because the intersection over i says that the read edge (x, y) accepts the witness edges whose profile is in slot r. The circuit makes $2n$ row projections at each of the $2k$ lengths, $4kn$ in all, and uses no converse. □

Proposition 1.2 (Domain size)**.** *For some constant $c > 0$, for every $k \ge 1$ and $n \ge ck4^k$, every* CoR *circuit computing G_k on all n-element structures has at least 2^{k-1} composition gates.*

Proof. By Corollary 1.8 with $\varepsilon = 0$, the lower bound holds at every domain size $n \ge n_0(2^k)$. Since

$$n_0(2^k) = \lceil 12 \cdot 4^k (k+1) \ln 2 \rceil + 3 \le 18k4^k$$

for all $k \ge 1$, we may take $C = 18$. □

4.4 Random inputs

Corollary 4.5 (Random inputs)**.** *Fix $k \ge 1$ and n. Color the edges of the complete graph on n vertices independently and uniformly with the colors in $2^{[k]}$, determine P_i, Q_i by the colors as in §1.3, and fix a vertex $\hbar$. Choose a color λ uniformly, a vertex v uniformly among those with $\chi(\hbar, v) = \lambda^c$ (any vertex if there is none), and a fair bit; if the bit is one, apply the fold of λ. Then every relational circuit with g composition gates answers $G_k(\hbar, v)$ correctly with probability at most $\frac{1}{2} + g/2^k + \delta$, where $\delta \coloneqq n^3 4^k \exp(-(n-3)/4^k)$, and for each $g \le 2^{k-1}$ some circuit with g composition gates answers correctly with probability at least $\frac{1}{2} + g/2^k - \delta$.*

Proof. Let $\mathcal{W}$ be the event that the coloring has the two-witness property. The proof of Lemma 3.1, with $N = 2^k$, gives $\Pr[\neg\mathcal{W}] \le \delta$. Fix a coloring in $\mathcal{W}$. Then $\hbar$ has edges of every color, so v is uniform among the vertices with $\chi(\hbar, v) = \lambda^c$. For each color λ, let $\mathrm{err}_{\lambda,0}, \mathrm{err}_{\lambda,1}$ indicate an error of

C at $(\hbar, v_{\lambda^c})$ on $\mathcal{M}_0$ and on $\mathcal{M}_\lambda$. The error-count inequality in the proof of Theorem 3.6, with the tests of Lemma 4.1 and Lemma 3.7, gives

$$\sum_{\lambda} (\mathrm{err}_{\lambda,0} + \mathrm{err}_{\lambda,1}) \geq 2^k - \mathrm{cost}(C) \geq 2^k - 2g$$

for every choice of the vertices v_μ of §3.1, hence also in expectation when each v_μ is uniform among the vertices with $\chi(\hbar, v_\mu) = \mu$. Given the coloring, the probability of an error is the average of $\mathrm{err}_{\lambda,b}$ over uniform λ and b, so it is at least $(2^k - 2g)/2^{k+1} = \frac{1}{2} - g/2^k$. Averaging over the colorings proves the upper bound.

For the lower bound, choose g of the pairs $\{\lambda, \lambda^c\}$ and keep, in the circuit of Proposition B.1, only the terms for these pairs. This circuit has g composition gates. On a coloring in $\mathcal{W}$, it is correct on both structures of the $2g$ tests whose colors lie in the chosen pairs, and for every other color it answers zero at the read edge, which is correct when the bit is one. Its success probability is therefore at least $(1 - \delta)(\frac{1}{2} + g/2^k) \geq \frac{1}{2} + g/2^k - \delta$. □

4.5 Reflexive–transitive closure

Proposition 1.7 (Closure does not help)**.** *Let C be a* CoR *circuit with closure gates $(-)^*$ and let $g_*(C)$ count its distinct closure gates. If $C \equiv_{\mathrm{fin}} G_k$, then $2g_;(C) + g_*(C) \geq 2^k$.*

Proof. On the reference structure, (CT) gives $X; X = \mathbf{1}$ whenever the off-diagonal support S_X is nonempty. Hence $X^* = \mathbf{1}$ if $S_X \neq \varnothing$ and $X^* = \mathbf{id}$ otherwise; in particular, closure preserves color-determined relations, as condition (i) of a test system requires. Let λ be a color, and put $J := \Lambda \setminus \{\lambda\}$. If $S_X = \varnothing$, then $X^{(\lambda)} = X$, and both $(X^{(\lambda)})^*$ and $(X^*)^{(\lambda)}$ equal $\mathbf{id}$. If S_X contains a color in J, the table of §3.4 gives $X^{(\lambda)}; X^{(\lambda)} = \mathbf{1}$, so $(X^{(\lambda)})^* = \mathbf{1} = (X^*)^{(\lambda)}$. Thus a closure gate notices the fold of λ only when $S_X = \{\lambda\}$, so it notices at most one fold. Theorem 3.6 and Lemma 3.7 give $2^k \leq m_1(C) + 2m_2(C) + g_*(C) \leq 2g_;(C) + g_*(C)$ with the tests of Lemma 4.1. □

5 Fixed vocabularies and model checking

The lower bound for G_k uses a growing input vocabulary. We now show that a single binary relation suffices for a superpolynomial composition lower bound, and that fixing the vocabulary changes the optimal circuit rate to $2^{\Theta(m/\log(m+2))}$. Throughout this section, m counts nodes in the *source syntax tree*; only the target may share subexpressions. Logarithms are to base two.

Theorem 5.1 (Fixed-vocabulary succinctness)**.** *For every sufficiently large m, there is an* FO^3 *query of syntax-tree size at most m over one binary relation such that every equivalent relational circuit over finite structures has at least $2^{\Omega(m/\log(m+2))}$ composition gates. Conversely, over any fixed finite binary vocabulary σ, every* FO^3 *formula of syntax-tree size m with at most two free variables has an equivalent relational circuit of total size $2^{O_\sigma(m/\log(m+2))}$, constructible within the same time bound. The upper-bound translation is correct on all nonempty structures.*

We first give the constructive upper bound, then transfer the main lower bound through a single-relation encoding. The final subsection derives the model-checking consequence.

5.1 The fixed-vocabulary upper bound

Fix a finite binary vocabulary σ. Since the variable names are restricted to x, y, z, formulas are trees over a fixed finite ranked alphabet. Consequently, for some constant $a_\sigma > 1$, there are at most a_σ^{t+1} syntactically distinct formulas of size at most t. It follows that d distinct formulas $\psi_1, \dots, \psi_d$ satisfy

$$\sum_{j=1}^{d} |\psi_j| \geq c_\sigma d \log(d+1)$$

for a constant $c_\sigma > 0$. Indeed, for all sufficiently large d, at most $d/2$ formulas have size below a sufficiently small constant times $\log d$. At least $d/2$ of the chosen formulas therefore have size $\Omega_\sigma(\log d)$; reducing c_σ handles the remaining values of d.

Proof of the upper bound in Theorem 5.1. Use the Boolean-shell translation of Appendix D, but share the translated children instead of copying them into each DNF clause. For a quantified subformula $Qw\,\psi$, its shell treats every atom and every maximal quantified subformula of ψ as a Boolean input. Let d be the number of *distinct* quantified children. Choosing one occurrence of each gives disjoint subtrees of the source, whose total size is at most m. The preceding inequality yields

$$d \log(d+1) = O_\sigma(m), \qquad d = O_\sigma\left(\frac{m}{\log(m+2)}\right).$$

There are only $O_\sigma(1)$ possible relation and equality atoms. Thus the shell is a Boolean function of $r = d + O_\sigma(1) = O_\sigma(m/\log(m+2))$ distinct inputs. Repeated occurrences of the same child are assigned the same input.

Process quantified subformulas from the inside out. By the binary-assignment convention of §2.1, each quantified child has at most two free variables and already has a binary representation. A unary child has a dummy coordinate, so its orientation can be changed by converse; a sentence is represented by $\mathbf{0}$ or $\mathbf{1}$. Reflexive atoms have constant-size representations as in §2.2.

The truth-table DNF of a shell has at most 2^r clauses and r literals per clause. At an existential quantifier, group the literals of a clause by their free-variable pairs, obtaining $A(u,v) \wedge B(u,w) \wedge D(w,v)$. Eliminating w gives $A \cap (B;D)$, with empty groups represented by $\mathbf{1}$. Thus each clause introduces at most one composition. Negated literals and universal quantifiers use complement. Each child circuit is computed once and reused in every clause: only $O((r+1)2^r)$ new gates, including at most 2^r compositions, are introduced at this shell.

There are at most m quantified occurrences. The unquantified root shell, whose free variables are contained in a pair, is translated directly by Boolean operations. All atomic representations together cost $O_\sigma(1)$ gates if shared. Hence both the total size and the composition count are bounded by

$$|C_\varphi| \leq \mathrm{poly}_\sigma(m)\, 2^{O_\sigma(m/\log(m+2))} = 2^{O_\sigma(m/\log(m+2))}.$$

The translation is correct on all nonempty structures by the same quantifier-elimination identity as the classical translation. Syntactic comparison of children, enumeration of shell truth assignments, and construction of the circuit take at most a polynomial factor in m times this bound, which is absorbed into the exponent. □

The counting step uses disjoint subtrees of the source. It therefore does not give the same upper bound when m instead measures a DAG-represented source formula. Nor does the construction guarantee a subexponential-size CoR term: unfolding the shared children can increase its size.

5.2 The lower bound over one binary relation

We encode the input relations of G_k in a single relation on copies of the original domain. The following block calculation preserves the composition measure even with unrestricted complement and sharing.

Lemma 5.2 (Block simulation)**.** *Suppose an encoding maps each structure with domain V to a structure on $V \times L$, where $|L| = q$, with one input relation E. Assume that each block $E_{ab}(u, v) \coloneqq E((u, a), (v, b))$ is an original input relation or one of $\mathbf{0}, \mathbf{1}, \mathbf{id}$, with the choice depending only on $a, b \in L$. Every relational circuit $C(E)$ can then be simulated over the original signature so as to compute any designated output block, using at most $q^3 g_;(C)$ composition gates.*

Proof. For every gate X, compute all q^2 relations $X_{ab}(u, v) \coloneqq X((u, a), (v, b))$. The blocks of $\mathbf{0}$ and $\mathbf{1}$ are respectively $\mathbf{0}$ and $\mathbf{1}$; the identity has block $\mathbf{id}$ when $a = b$ and $\mathbf{0}$ otherwise. Boolean operations act blockwise, including complement relative to V^2, and converse exchanges the layer indices:

$$(\overline{X})_{ab} = \overline{X_{ab}}, \qquad (X^\smile)_{ab} = (X_{ba})^\smile.$$

Composition satisfies

$$(X; Y)_{ab} = \bigcup_{c \in L} (X_{ac}; Y_{cb}).$$

Thus each composition gate introduces at most q^3 compositions over all output blocks; the other operations introduce none. The blocks of a shared gate are computed once and reused. Induction over the circuit proves correctness. □

The encoding. List the $2k$ input symbols of G_k as $R_1, \ldots, R_{2k}$, with $R_i \coloneqq P_i$ and $R_{k+i} \coloneqq Q_i$. Put

$$\ell \coloneqq \lceil \log(2k) \rceil, \qquad L \coloneqq \{0,1\}^{\leq \ell}, \qquad q \coloneqq |L| = 2^{\ell+1} - 1 < 8k,$$

and choose distinct leaves $w_1, \ldots, w_{2k} \in \{0,1\}^\ell$. For a structure $\mathcal{M}$ on V, the encoded structure $\mathrm{Enc}(\mathcal{M})$ has domain $V \times L$. Its sole relation E consists of the following edges:

$$\begin{aligned}
&((u,t),(u,tb)) && (u \in V,\ |t| < \ell,\ b \in \{0,1\}),\\
&((u,t),(u,t)) && (u \in V,\ t \neq \epsilon,\ t \text{ ends in } 1),\\
&((u,w_j),(v,\epsilon)) && ((u,v) \in R_j^{\mathcal{M}},\ 1 \leq j \leq 2k).
\end{aligned}$$

There are no other edges. Each u has a binary routing tree; loops mark its 1-children, and the edges from leaf w_j back to the root layer store R_j. Every layer-to-layer block is $\mathbf{0}$, $\mathbf{id}$, or some R_j, so Lemma 5.2 applies. The encoded graphs are directed and may have loops.

Three-variable decoders. Let

$$N(u, v) \coloneqq E(u, v) \wedge u \neq v, \qquad L_0(v) \coloneqq \neg E(v, v), \qquad L_1(v) \coloneqq E(v, v).$$

For distinct variable names $u, v \in \{x, y, z\}$, define

$$\theta_\epsilon(u, v) \coloneqq N(u, v), \qquad \theta_{bs}(u, v) \coloneqq \exists w \big(N(u, w) \wedge L_b(w) \wedge \theta_s(w, v)\big),$$

where w is the third variable name. Inside $\theta_s(w, v)$, the old name u may be rebound; all such bindings are local to that subformula. For every word t, expanding θ_t gives a genuine FO^3 formula of size $\Theta(|t| + 1)$.

Starting at (u, ϵ), the first ℓ non-loop steps of θ_{w_j} uniquely follow the word w_j through the routing tree. Its final non-loop edge returns to the root layer and reads R_j. Consequently,

$$\mathrm{Enc}(\mathcal{M}) \models \theta_{w_j}((u, \epsilon), (v, \epsilon)) \quad \Longleftrightarrow \quad \mathcal{M} \models R_j(u, v).$$

Furthermore, each selected decoder θ_{w_j} starting at the root layer can hold only when its second argument is also in that layer. The inequalities in N exclude the marking loops from both the routing steps and the final data step; data edges can still encode loops of the original relations, since their two layer indices differ.

Proof of the lower bound in Theorem 5.1. Replace each relation-atom occurrence $R_j(u, v)$ in G_k by $\theta_{w_j}(u, v)$, obtaining $\widehat{G}_k$ over the single symbol E. Each decoder uses only three variable names. Any rebinding of the third name is local to that atom replacement, so it does not affect another conjunct or disjunct of G_k. There are $4k$ relation-atom occurrences, hence

$$m_k := |\widehat{G}_k| = \Theta(k \log(k + 1)).$$

When x, y lie in the root layer, every successful decoded atom on (x, z) forces z into the root layer as well. Since $k \geq 1$,

$$\mathrm{Enc}(\mathcal{M}) \models \widehat{G}_k((u, \epsilon), (v, \epsilon)) \quad \Longleftrightarrow \quad \mathcal{M} \models G_k(u, v).$$

Let C compute $\widehat{G}_k$ on all finite E-structures. Lemma 5.2 produces a circuit $C^{\downarrow}$ over the original signature computing its root–root output block, with $g_{;}(C^{\downarrow}) \leq q^3 g_{;}(C)$. The displayed equivalence gives $C^{\downarrow} \equiv_{\mathrm{fin}} G_k$, so Theorem 1.1 yields

$$2^{k-1} \leq g_{;}(C^{\downarrow}) \leq q^3 g_{;}(C), \qquad g_{;}(C) \geq \frac{2^{k-1}}{q^3} > \frac{2^{k-1}}{(8k)^3}.$$

Since $k = \Theta(m_k / \log(m_k + 2))$, this is $2^{\Omega(m_k / \log(m_k+2))}$. For every sufficiently large size bound m, choose $k = \lfloor cm / \log(m + 2) \rfloor$ with a sufficiently small absolute constant $c > 0$. Then $m_k \leq m$ and the same estimate gives the stated $2^{\Omega(m / \log(m+2))}$ lower bound. No well-formedness test for the encoding is needed: correctness on all finite E-structures already includes the encoded ones. □

For any fixed vocabulary containing at least one binary symbol, the same lower bound follows by fixing all other input relations to $\mathbf{0}$. Together with the upper bound, this determines both the worst-case composition count and the worst-case total circuit size at the scale $2^{\Theta(m / \log(m+2))}$. The composition lower bound also holds for terms, but does not determine their optimal general upper bound.

5.3 Model checking on directed graphs

Corollary 5.3. *Fix a finite binary vocabulary σ and $\eta > 0$. A tree-represented $\mathrm{FO}^3[\sigma]$ sentence of size m can be evaluated on an n-element structure in*

$$2^{O_\sigma(m / \log(m+2))}\, n^{\omega+\eta}$$

time, including compilation. In particular, this holds for directed graphs with one input edge relation.

Proof. Construct the circuit of Theorem 5.1 and evaluate each gate once in topological order. A composition is a Boolean matrix product, computable in $O(n^{\omega+\eta})$ time for any fixed $\eta > 0$; all other gates take $O(n^2)$ time. By the assignment semantics of §2.1, a sentence denotes either $\mathbf{1}$ or $\mathbf{0}$, so any output entry gives its truth value. The circuit construction time and input-reading time are absorbed in the displayed bound. □

Conditional tightness. For fixed vocabularies, this improves the $2^{O(m)}$ formula-size factor in [NU27, Theorem 10]. Under the low-dimensional 3-OV hypothesis, their Theorem 72 (Appendix T) rules out $2^{o(m/\log(m+2))}n^{3-\delta}$ time for every fixed $\delta > 0$, even for tree-represented existential-positive sentences over one binary relation. Their bound counts edges M rather than vertices. Retaining at most three isolated vertices preserves FO^3 sentences and leaves at most $2M + 3$ vertices [NU27, Proposition 6], giving the stated vertex bound. Building an adjacency matrix takes $O((M+1)^2)$ time, absorbed after reducing δ to lie in $(0, 1)$. Thus $m/\log m$ is the conditionally optimal exponent scale for truly subcubic evaluation, without a claim of optimality for ω.

6 Two-variable logic and tree size

The formula G_k is FO^2-definable, but its displayed linear-size form uses three variable names. This section quantifies the difference, proves the term-size bound of Theorem 6.3, and separates CoR from FO^2 (§6.4).

6.1 Quantifiers and fold sensitivity

Recall the linear translation τ from FO^2 formulas into CoR given in §2.2. On the reference structure, the off-diagonal support of $R \cap \mathbf{id}$ is empty. Therefore neither of the compositions used to represent a reflexive atom, $(R \cap \mathbf{id}); \mathbf{1}$ or $\mathbf{1}; (R \cap \mathbf{id})$, notices a fold (see Lemma 3.5). Every quantifier, by contrast, contributes one *one-sided* composition $X; \mathbf{1}$ or $\mathbf{1}; X$; by the same lemma, it can notice at most one fold. Boolean operations notice none. This gives a bound in terms of quantifiers, rather than all composition occurrences in $\tau(\varphi)$.

Lemma 6.1 (Quantifier cost)**.** *On the reference structure, every* FO^2 *formula* φ *satisfies*

$$\mathrm{cost}(\tau(\varphi)) \leq \#_{\exists\forall}(\varphi).$$

The bound holds without restricting the Boolean connectives or the quantifier alternation of the target formula.

Combining Lemma 6.1 with Theorem 3.6 and the tests of Lemma 4.1 gives

$$\varphi \equiv_{\mathrm{fin}} G_k \Longrightarrow \#_{\exists\forall}(\varphi) \geq 2^k.$$

The FO^2 expansion obtained from the DNF of §1.1 uses exactly 2^k quantifiers, so the optimum quantifier count for G_k is exactly 2^k. No restriction to existential target formulas is made.

6.2 Counting input occurrences in trees

The test–cost theorem also yields a stronger bound for syntax trees. A *singleton selector* for a color λ is a Boolean formula in the tables of the input relations, with constants allowed, whose support on the entire palette is exactly $\{\lambda\}$.

Lemma 6.2 (Disjoint input-occurrence charging)**.** *Fix the reference structure and its folds. Suppose every singleton selector requires at least $d > 0$ input-relation occurrences. Then every* CoR *term T, whether correct or not, satisfies*

$$\mathrm{cost}(T) \le \#_{\mathrm{rel}}(T)/d.$$

Appendix B.1 proves the lemma by assigning disjoint sets of input occurrences to composition nodes. The disjointness uses the tree structure of a term.

For the input relations of §4.1, a Boolean formula in their tables that contains neither P_i nor Q_i takes the same value on a color and on its i-th-bit flip. Hence a singleton selector contains P_i or Q_i for every i, so $d = k$, and this is exact, since H_λ selects λ. Theorem 3.6 and Lemma 6.2 give $\#_{\mathrm{rel}}(T) \ge k2^k$ for every CoR term $T \equiv_{\mathrm{fin}} G_k$. By the tree-size inequality from §2.3,

$$|T| \ge 2k2^k - 1 \qquad (T \equiv_{\mathrm{fin}} G_k),$$

which is the lower bound of the following theorem.

Theorem 6.3 (Term size)**.** *For every $k \ge 1$, every* CoR *term $T \equiv_{\mathrm{fin}} G_k$ has at least $2k2^k - 1$ nodes, and G_k is equivalent to a* CoR *term with $(2k+6)2^k - 5$ nodes.*

Proof. The lower bound is the inequality above. Factoring the conditions on the read edge into a binary decision tree gives a term with $(2k+6)2^k - 5$ nodes that is equivalent to G_k on all nonempty structures (Appendix B.2). □

6.3 Sentences and the variable hierarchy

The preceding subsections establish the binary-query bounds of Theorem 1.4. We now prove its sentence extension for $G_k^\forall = \forall x\, \forall y\, (x = y \vee G_k(x, y))$.

Theorem 1.4. *For every $k \ge 1$, every* FO^2 *formula equivalent to $G_k(x, y)$ over finite structures has at least 2^k quantifiers.*

Quantitative and sentence extensions. We also prove two refinements. Every FO^2 formula defining G_k has at least $k2^k$ relation-atom occurrences, and its minimum syntax-tree size is $\Theta(k2^k)$. Moreover, $G_k^\forall$ is an FO^2-definable FO^3 sentence of size $O(k)$ and quantifier depth three, but every equivalent FO^2 sentence over finite structures has at least 2^k quantifiers and $k2^k$ relation-atom occurrences; its minimum FO^2 tree size is $\Theta(k2^k)$.

Proof of the quantitative and sentence extensions. The exact 2^k-quantifier lower bound for the binary query G_k was established in §6.1. For any equivalent FO^2 formula φ, apply Theorem 3.6 and Lemma 6.2 to $\tau(\varphi)$. By the relation-atom preservation established in §2.2, this gives $\#_{\mathrm{rel}}(\varphi) \ge k2^k$. The FO^2 DNF expansion from §1.1 has $O(k2^k)$ nodes and exactly 2^k quantifiers, proving the tree-size claim for the binary query.

For the sentence $G_k^\forall$, the source has size $8k + 4$ and quantifier depth three. By Lemma 4.1, it is true in $\mathcal{M}_0$ and false in every $\mathcal{M}_\lambda$. A sentence is represented by a constant binary relation, and either constant is its own prediction. Thus these structures give 2^k tests for the sentence itself. For any equivalent FO^2 sentence φ, apply Theorem 3.6 to $\tau(\varphi)$. Lemma 6.1 gives $\#_{\exists\forall}(\varphi) \ge 2^k$, and Lemma 6.2, together with exact preservation of $\#_{\mathrm{rel}}$, gives $\#_{\mathrm{rel}}(\varphi) \ge k2^k$. Conversely, substitute

the FO^2 expansion of G_k into the sentence. It has $2^k + 2$ quantifiers and $O(k2^k)$ nodes, proving the sentence upper bounds and the matching order of tree size. □

The quantifier optimum for this sentence is therefore in $[2^k, 2^k + 2]$; the exact 2^k optimum proved earlier concerns the binary query.

This complements the variable-saving separations of Vinall-Smeeth [VS24, Theorem 2], whose stated range starts at three variables on the target side. Our result concerns the remaining comparison of three and two variables, over growing binary signatures. It does not claim a lower bound for separating a single prescribed pair of structures as a function of their size. On pure finite linear orders, in contrast, Grohe and Schweikardt [GS05, Theorem 4.3] give a polynomial translation from three-variable sentences to two-variable sentences. The qualifications "pure orders" and "sentences" are essential to that comparison.

6.4 Separating CoR from FO^2

The linear translation of §2.2 goes from FO^2 to CoR. On the reference structure, a composition notices at most two folds and a quantifier at most one, so tests on the reference structure bound CoR circuits and FO^2 formulas alike, up to a factor of two. To separate the two formalisms, we use a structure on which a quantifier still notices at most one fold, but a single composition notices all of them.

Use input symbols $R, P_1, \dots, P_k$, and put

$$\mathscr{P}_i^1 \coloneqq \mathbf{id} \cap P_i, \qquad \mathscr{P}_i^0 \coloneqq \mathbf{id} \cap \overline{P_i}, \qquad U_k \coloneqq R; \bigcap_{i=1}^{k} \left[(\mathscr{P}_i^0; \mathbf{1}; \mathscr{P}_i^0) \cup (\mathscr{P}_i^1; \mathbf{1}; \mathscr{P}_i^1) \right].$$

The complement in $\mathscr{P}_i^0$ is taken before intersection with the identity; it is not the complement of $\mathscr{P}_i^1$. Expanding these abbreviations gives an $O(k)$-size tree. For a vertex v, call $\pi(v) \coloneqq \{i \in [k] : P_i(v, v)\}$ its *loop pattern*. Then $U_k(x, y)$ holds exactly when some R-successor z of x satisfies $\pi(z) = \pi(y)$, that is, when

$$\varphi_k(x, y) \coloneqq \exists z \left(R(x, z) \wedge \bigwedge_{i=1}^{k} (P_i(z, z) \leftrightarrow P_i(y, y)) \right)$$

holds. Here $\alpha \leftrightarrow \gamma$ abbreviates $(\alpha \wedge \gamma) \vee (\neg\alpha \wedge \neg\gamma)$.

The structures. Let $V \coloneqq 2^{[k]} \times \mathbb{Z}_{11}$ and $\Gamma \coloneqq \{1, 2, 3, 4, 7, 8\} \subseteq \mathbb{Z}_{11}$. The structure $\mathcal{W}_0$ on V has the loops $P_i((A, s), (A, s))$ for $i \in A$, no other P_i-entries, and $R((A, s), (B, s'))$ exactly when $s' - s \in \Gamma$; thus $\pi(A, s) = A$, and R has no loops. The *atomic type* of a pair (u, v) of distinct vertices is $(R(u, v), R(v, u), \pi(u), \pi(v))$, and that of a loop (u, u) is $\pi(u)$; it determines the values of all atomic formulas at the pair. As $s' - s$ ranges over $\mathbb{Z}_{11} \setminus \{0\}$, the truth values of $s' - s \in \Gamma$ and $s - s' \in \Gamma$ take each of the four combinations at least twice: $(1, 0)$ at $s' - s \in \{1, 2\}$, $(0, 1)$ at $s' - s \in \{9, 10\}$, $(1, 1)$ at $s' - s \in \{3, 4, 7, 8\}$, and $(0, 0)$ at $s' - s \in \{5, 6\}$. Hence every vertex u is the first entry of at least two pairs of each atomic type $(r, r', \pi(u), B)$, and the second entry of at least two pairs of each atomic type $(r, r', B, \pi(u))$.

Fix the hub $\hbar \coloneqq (\varnothing, 0)$. For $A \subseteq [k]$, the fold of A yields the structure $\mathcal{W}_A$ obtained from $\mathcal{W}_0$ by removing from R the edges $(\hbar, v)$ with $v \neq \hbar$ and $\pi(v) = A$. It changes the atomic types of these pairs and of their converses, and of no other pair. Afterwards, no pair with first entry $\hbar$ has a type $(1, r', \varnothing, A)$, and no pair with second entry $\hbar$ has a type $(r, 1, A, \varnothing)$; all other types still occur at $\hbar$.

Type tables and predictions. A relation $X \subseteq V^2$ has a *type table* if $X(u,v)$ depends only on the atomic type of (u,v) in $\mathcal{W}_0$; let S_X be the set of atomic types at which X holds. Every atomic type of $\mathcal{W}_A$ occurs in $\mathcal{W}_0$, so the prediction $X^{(A)}$ can read the same table at the atomic types of $\mathcal{W}_A$. As in §3.3, an operation op notices the fold of A at operand values $\mathbf{U}$ if $\mathrm{op}(\mathbf{U}^{(A)}) \neq \mathrm{op}(\mathbf{U})^{(A)}$, and Boolean operations never notice a fold.

View every FO^2 formula as a binary assignment relation, as in §2.1; its quantifiers act as the projections $\exists y\, \psi \equiv \psi; \mathbf{1}$ and $\exists x\, \psi \equiv \mathbf{1}; \psi$. By induction on the formula, every relation that an FO^2 formula defines on $\mathcal{W}_0$ has a type table: if X has one, then $(X;\mathbf{1})(u,v)$ holds exactly when S_X contains the atomic type of some pair with first entry u, and these types are determined by $\pi(u)$; symmetrically for $\mathbf{1}; X$.

Lemma 6.4 (Projections notice at most one fold)**.** *If X has a type table, then $X;\mathbf{1}$ and $\mathbf{1};X$ each notice the fold of at most one $A \subseteq [k]$.*

Proof. At a pair (u,v), the prediction of $X;\mathbf{1}$ holds exactly when S_X contains the atomic type in $\mathcal{W}_0$ of some pair with first entry u, and $X^{(A)};\mathbf{1}$ holds exactly when S_X contains the atomic type in $\mathcal{W}_A$ of some such pair. If $u \neq \hbar$, the fold changes at most one pair with first entry u, namely $(u,\hbar)$; since every type occurs at least twice among these pairs in $\mathcal{W}_0$, the two sets of types coincide. If $u = \hbar$, the fold removes exactly the types $(1, r', \varnothing, A)$ and adds none. Hence $X;\mathbf{1}$ notices the fold of A only if the types in S_X of pairs with first entry $\hbar$ are all of the form $(1, r', \varnothing, A)$, and there is at least one; this determines A. The argument for $\mathbf{1};X$ uses second entries. □

Proposition 1.5. *For every $k \geq 1$, every FO^2 formula equivalent to U_k over finite structures has at least 2^k quantifiers.*

Equational sentence extension. The CoR equation $\mathbf{id} \cup U_k = \mathbf{1}$ is FO^2-definable as a sentence, but every equivalent FO^2 sentence over finite structures has at least 2^k quantifiers.

Proof of the query and sentence claims. For the upper bounds, let $\vartheta_A(u)$ be the quantifier-free formula asserting that $\pi(u) = A$. The FO^2 formula

$$\bigvee_{A \subseteq [k]} \left[\vartheta_A(y) \wedge \exists y\, (R(x,y) \wedge \vartheta_A(y))\right]$$

defines U_k with exactly 2^k quantifiers; the inner y is rebound only in the second conjunct. The equation $\mathbf{id} \cup U_k = \mathbf{1}$ says that for all x and all $y \neq x$, some R-successor of x has the loop pattern of y, and so does the FO^2 sentence

$$\forall x \bigwedge_{A \subseteq [k]} \left[\exists y\, (x \neq y \wedge \vartheta_A(y)) \to \exists y\, (R(x,y) \wedge \vartheta_A(y))\right].$$

For the lower bounds, every vertex of $\mathcal{W}_0$ has an R-successor with each loop pattern, so U_k is the full relation on $\mathcal{W}_0$ and the equation holds there. On $\mathcal{W}_A$, the hub has no R-successor with loop pattern A, so $U_k(\hbar, v)$ fails for every $v \neq \hbar$ with $\pi(v) = A$, and the equation fails. Let ψ be an FO^2 formula defining U_k, or an FO^2 sentence equivalent to the equation. Its value on $\mathcal{W}_0$ is the full relation, whose prediction on $\mathcal{W}_A$ is again full, but its value on $\mathcal{W}_A$ is not. If no quantifier of ψ noticed the fold of A, then, as in the proof of Lemma 3.4, every subformula would take its predicted

value on $\mathcal{W}_A$, by induction on the formula: atomic types determine atomic formulas, and Boolean operations notice nothing. Hence some quantifier notices the fold of each A, and by Lemma 6.4 each quantifier notices at most one, so ψ has at least 2^k quantifiers. □

7 Scalar aggregation and matrix query languages

We extend the fold argument from binary relations to semiring-valued matrix computations. We first describe the short three-attribute expression, then the matrix circuit model, and finally the fold sensitivity bounds that force exponential arity-elimination overhead.

7.1 The three-attribute query and its matrix interpretation

Let $\mathfrak{S} \coloneqq (S, \oplus, \otimes, 0_S, 1_S)$ be a commutative semiring and V a finite common index set, with inputs $P_i, Q_i : V^2 \to S$. An *annotated relation* assigns an S-value to each tuple. In positive annotated relational algebra, union adds annotations, natural join multiplies them, and projection sums over eliminated attributes; renaming changes attribute names. ARA(3) restricts intermediate relations to at most three attributes, analogous to the ternary assignment relations of FO^3 (§2.1). Over Boolean annotations it corresponds to positive three-variable computation.

For an explicit ARA(3) expression, let P_i^{xy} denote P_i with attributes (x, y), and similarly for P_i^{xz}, Q_i^{xy}, and Q_i^{xz}. Our semiring-valued query is

$$\mathcal{G}_k \coloneqq \pi_{xy}\left(\bigbowtie_{i=1}^{k}\left[(P_i^{xy} \bowtie Q_i^{xz}) \cup (Q_i^{xy} \bowtie P_i^{xz})\right]\right).$$

Every intermediate relation has at most the three attributes (x, y, z): the k conditions are combined with the *same* z, which is eliminated only at the final projection. Thus the ARA(3) expression has size $O(k)$. Equivalently, its scalar value is

$$\mathcal{G}_k(x, y) = \bigoplus_{z \in V} \bigotimes_{i=1}^{k}\left[(P_i(x, y) \otimes Q_i(x, z)) \oplus (Q_i(x, y) \otimes P_i(x, z))\right].$$

For $S = \mathbb{B}$ with $\oplus = \vee$ and $\otimes = \wedge$, this is precisely $G_k(x, y)$, without negation or universal quantification. For integer min–plus, take $\mathfrak{S} \coloneqq (\mathbb{Z} \cup \{+\infty\}, \min, +, +\infty, 0)$; then the same query becomes

$$\mathcal{G}_k(x, y) = \min_{z \in V} \sum_{i=1}^{k} \min\{P_i(x, y) + Q_i(x, z),\ Q_i(x, y) + P_i(x, z)\}.$$

An explicit MATLANG expression. Although MATLANG retains only matrices, vectors, and scalars, the DNF of §1.1 gives an expression for $\mathcal{G}_k$ using matrix–vector products. For $A \subseteq [k]$, let H_A be the pointwise semiring analogue of the relation in §1.1:

$$H_A(x, y) \coloneqq \bigotimes_{i \in A} P_i(x, y) \ \otimes \bigotimes_{i \in A^c} Q_i(x, y),$$

where an empty product is 1_S. Let $\mathbf{1}_S$ be the constant column vector with every entry 1_S. Computing the column vector r_A and then broadcasting it along the rows gives

$$r_A \coloneqq H_{A^c}\mathbf{1}_S, \qquad \mathcal{G}_k(x, y) = \bigoplus_{A \subseteq [k]} H_A(x, y) \otimes r_A(x). \tag{1}$$

Distributivity expands the k clauses and interchanges the finite aggregations over A and z, proving the identity for arbitrary S-valued inputs. Each r_A uses one matrix–vector product; all other steps are entrywise operations or row broadcasting. Thus 2^k such products suffice, even without matrix–matrix products. Our lower bound shows that an exponential number of data-index aggregations remains necessary even if both kinds of products, other permitted operations, and sharing are allowed.

7.2 Typed matrix circuits

In the idempotent case, write semiring addition as $\vee$. The circuit model below permits arbitrary binary kernels, rather than only semiring multiplication; it therefore includes the corresponding MATLANG operations. This strengthening is useful because the lower bound charges only data-index aggregations.

We use the dimension symbols n and 1. Intermediate arrays have shapes $n \times n$, $n \times 1$, $1 \times n$, or 1×1—matrices, column vectors, row vectors, and scalars. A data-index contraction of an $a \times n$ and an $n \times c$ array is

$$Z(i,j) \coloneqq \bigvee_{z=1}^{n} \kappa(X(i,z), Y(z,j)), \qquad a, c \in \{1, n\},$$

where the kernel $\kappa : S \times S \to S$ may vary by gate and need not be associative, monotone, or zero-preserving. Contractions over an index of size one apply the kernel without aggregating over data, and are not charged. We also allow arbitrary uniform entrywise functions, transpose, scalar and vector broadcasting, diagonal extraction and formation, uniform constant vectors, and a diagonal mask with two fixed values, one on and one off the diagonal. These operations are not charged. Standard MATLANG operations are described in [BGVW19, BGV22].

All circuits have fixed directed acyclic wiring. Functions, kernels, and uniform scalar constants are fixed before the input is seen, though they may depend on n and k in a size-specific circuit. "Uniform" means that the same entrywise function is applied at every position of its type and cannot access vertex names. A kernel receives only its two scalar arguments, not the output position (i, j), the contracted index z, a third matrix entry, or an encoding of the entire input unless computed by permitted gates. Data-dependent values and tests are allowed, but there is no primitive for data-dependent rewiring or arbitrary indexed access. The input arrays are square $n \times n$ matrices; no order, vertex identifiers, arbitrary input vectors, or vertex-dependent constant arrays are supplied for free.

Write $\#_{\mathrm{mv}}(C)$ for the number of matrix–vector and vector–matrix data-index contractions and $\#_{\mathrm{mm}}(C)$ for the number of matrix–matrix data-index contractions, counting each distinct gate once. All other operations, including vector inner products, are free in the resource measure. We next show that these free operations notice no folds, even though some of them may require compositions when expressed in CoR.

7.3 Fold sensitivity and finite join breadth

Let $(S, \vee, 0_S)$ be a join-semilattice with least element 0_S. Its *join breadth* β is the least nonnegative integer such that every finite join equals the join of at most β of its summands (the empty join is 0_S). We assume $\beta < \infty$. For a totally ordered semiring with min or max as addition, $\beta = 1$: one

summand attains the aggregate. In a product with b independent coordinates, different summands can determine the result in different coordinates, and breadth can be b.

The significance for folds is immediate. Each fold removes one color from the hub. If a join is already determined by at most β summands, removing a color outside those summands cannot change its value. We now make this observation precise for the three kinds of output pairs involving the hub.

Let V be the common finite index set of the input matrices and $n \coloneqq |V|$. For color-determined square arrays X, Y and any kernel $\kappa : S \times S \to S$, put

$$(X \mathbin{;_\kappa} Y)(u, v) \coloneqq \bigvee_{z \in V} \kappa(X(u, z), Y(z, v)).$$

Kernels may differ between gates; no order or monotonicity assumption is made on κ. Reference arrays are color-determined as in §3.2, now with entries $X[\mathbf{id}], X[\mu] \in S$. Their predictions read the same table at the folded coloring χ_λ. Entrywise functions and transpose notice no fold. For $J \subseteq \Lambda$, define

$$\kappa_J^{\mathrm{L}} \coloneqq \bigvee_{\mu \in J,\, \nu \in \Lambda} \kappa(X[\mu], Y[\nu]), \qquad \kappa_J^{\mathrm{R}} \coloneqq \bigvee_{\mu \in J,\, \nu \in \Lambda} \kappa(X[\nu], Y[\mu]),$$
$$\kappa_J^{\mathrm{D}} \coloneqq \bigvee_{\mu \in J} \kappa(X[\mu], Y[\mu]), \qquad \kappa_* \coloneqq \kappa_\Lambda^{\mathrm{L}} = \kappa_\Lambda^{\mathrm{R}}.$$

The reference product $Z \coloneqq X \mathbin{;_\kappa} Y$ has the table

$$Z[\mathbf{id}] = \kappa(X[\mathbf{id}], Y[\mathbf{id}]) \vee \kappa_\Lambda^{\mathrm{D}},$$
$$Z[\mu] = \kappa(X[\mathbf{id}], Y[\mu]) \vee \kappa(X[\mu], Y[\mathbf{id}]) \vee \kappa_*.$$

The calculation of §3.4 remains valid, with endpoint contributions formed by κ and, away from the hub, outside-witness contributions κ_* off the diagonal and $\kappa_\Lambda^{\mathrm{D}}$ on it. For the fold of λ, put $J \coloneqq \Lambda \setminus \{\lambda\}$. Outside-witness contributions in the hub row, hub column, and hub diagonal become κ_J^{L}, κ_J^{R}, and κ_J^{D}, respectively; their predictions use $J = \Lambda$. This follows from the same pairs of witness colors, while idempotence makes their multiplicities irrelevant.

Lemma 7.1 (Scalar local bounds)**.** *A scalar aggregation gate notices at most 3β folds. If one operand is constant across off-diagonal colors, it notices at most β; if both are, it notices none. For Boolean scalars with arbitrary kernels, the general bound improves to two.*

Proof. By the calculation above, the gate notices the fold of λ only if one of the joins κ_J^{L}, κ_J^{R}, and κ_J^{D} with $J = \Lambda \setminus \{\lambda\}$ differs from its value at $J = \Lambda$. Choose at most β summands of $\kappa_\Lambda^{\mathrm{L}}$ whose join is $\kappa_\Lambda^{\mathrm{L}}$, and let K^{L} be the set of their row colors, the row color of $\kappa(X[\mu], Y[\nu])$ being μ. If $\lambda \notin K^{\mathrm{L}}$, all chosen summands also occur in κ_J^{L}, so $\kappa_\Lambda^{\mathrm{L}} \le \kappa_J^{\mathrm{L}} \le \kappa_\Lambda^{\mathrm{L}}$. Define K^{R} in the same way from the column colors of summands of $\kappa_\Lambda^{\mathrm{R}}$, and K^{D} from the colors of summands of $\kappa_\Lambda^{\mathrm{D}}$. Hence the gate notices the fold of λ only if $\lambda \in K^{\mathrm{L}} \cup K^{\mathrm{R}} \cup K^{\mathrm{D}}$, a set of at most 3β colors.

If $Y[\mu]$ is independent of μ, then $\kappa_J^{\mathrm{L}} = \kappa_J^{\mathrm{D}}$ and $\kappa_J^{\mathrm{R}} = \kappa_*$ for every nonempty J; only K^{L} is needed. The case of X is symmetric. If both are constant, all three joins are unchanged. For Boolean scalars, if $\kappa_\Lambda^{\mathrm{D}} = 1$, a single color μ with $\kappa(X[\mu], Y[\mu]) = 1$ supplies a summand of value one to all three joins whenever $\mu \in J$, so only the fold of μ can be noticed. Otherwise the diagonal join is always zero, and one color for each of the other two joins suffices. □

The displayed reference table is color-determined, so these calculations also establish the closure required by Theorem 3.6.

On the reference structure, every vector is constant over its data index, and every scalar is a single value. Encode a column vector $\mathbf{c}$ as the square array $(u, v) \mapsto \mathbf{c}(u)$, a row vector $\mathbf{r}$ as $(u, v) \mapsto \mathbf{r}(v)$, and a scalar as a constant square array. Encoded reference vectors are constant across all edge colors and on the diagonal and are unchanged by prediction. Diagonal formation and extraction, broadcasting, and permitted constants notice no folds. Matrix–vector and vector–matrix contractions notice at most β folds by Lemma 7.1; a vector inner product notices none, since both operands are constant. Products over an index of size one also notice none.

Lemma 7.2 (Cost of typed products)**.** *Every typed matrix circuit C in this model satisfies*

$$\operatorname{cost}(C) \le \beta(\#_{\mathrm{mv}}(C) + 3\#_{\mathrm{mm}}(C)).$$

For Boolean scalars and arbitrary kernels, the bound improves to $\#_{\mathrm{mv}}(C) + 2\#_{\mathrm{mm}}(C)$.

Proof. Sum Lemma 7.1 over the distinct aggregation gates. All other permitted gates notice none, as shown above. □

Operation	Folds noticed, at most
Uniform entrywise function, transpose, permitted constant	0
Diagonal formation or extraction, broadcasting	0
Product over an index of size one	0
Vector inner product	0
Matrix–vector or vector–matrix contraction	β
Square matrix–matrix contraction	3β

The cost counts noticed folds, not the time needed to evaluate a gate.

Why join breadth matters. The factor β in the local matrix–vector bound cannot in general be replaced by a constant. For $b \ge 1$, the finite-set semiring $(2^{[b]}, \cup, \cap)$ has join breadth exactly b: every union is determined by at most b sets, whereas the union of the b singletons requires all of them. The same is true of $\mathsf{Trop}^b_{\mathbb{Z}}$: different coordinates can attain their minima at different summands. Moreover, when $2^{k-1} \ge b$, choose distinct colors $\lambda_1, \dots, \lambda_b$ with no two complementary, and put $X[\lambda_j] = \{j\}$ and $X[\mu] = \varnothing$ for all other colors and on the diagonal. Matrix–vector aggregation of X against the constant column $[b]$, using union for aggregation and intersection as kernel, yields $[b]$ at the hub on the reference structure. Folding λ_j changes this value to $[b] \setminus \{j\}$. Thus one aggregation notices b folds. This is local tightness, not a claim that the overall circuit bound is sharp in β.

7.4 Exponential arity-elimination lower bounds

We now apply Theorem 3.6 to the 2^k original fold tests. The inputs are encoded by the two distinct semiring values $0_S, 1_S$. On every test, idempotence makes $\mathcal{G}_k$ return 1_S or 0_S according to the Boolean truth of G_k, so each fold still changes the answer at the unchanged read edge. For natural and integer min–plus the two values are $+\infty$ and 0, respectively. Intermediate gates, however, may take arbitrary values of S, which is why the scalar analysis of the previous subsection is needed.

Theorem 1.6 (Tropical arity-elimination overhead)**.** *For every $k \geq 1$ and $\mathfrak{S} \in \{\mathsf{Trop}_{\mathbb{N}}, \mathsf{Trop}_{\mathbb{Z}}\}$, the query $\mathcal{G}_k$ has an ARA(3) expression of size $O(k)$, but every typed matrix circuit C over $\mathfrak{S}$ computing it on all finite inputs satisfies $\#_{\mathrm{mv}}(C) + 3\#_{\mathrm{mm}}(C) \geq 2^k$.*

Proof. Both tropical semirings have join breadth $\beta = 1$. Use the 2^k fold tests of Lemma 4.1, encoded as $0_S, 1_S$. Theorem 3.6 and Lemma 7.2 give $2^k \leq \#_{\mathrm{mv}}(C) + 3\#_{\mathrm{mm}}(C)$. The ARA(3) upper bound was constructed in §7.1. □

Theorem 7.3 (Exponential arity-elimination overhead)**.** *Let S be a nontrivial commutative idempotent semiring of finite join breadth β. For every $k \geq 1$, the query $\mathcal{G}_k$ has a positive ARA(3) expression of size $O(k)$, but every typed matrix circuit C computing it on all finite inputs satisfies*

$$\beta(\#_{\mathrm{mv}}(C) + 3\#_{\mathrm{mm}}(C)) \geq 2^k.$$

In particular, at least $2^k/(3\beta)$ aggregation gates are necessary. Conversely, 2^k matrix–vector products suffice.

Proof. The $0_S, 1_S$ encodings above supply 2^k fold tests. Theorem 3.6 and Lemma 7.2 give the lower bound. Since $\#_{\mathrm{mv}}(C) + 3\#_{\mathrm{mm}}(C) \leq 3(\#_{\mathrm{mv}}(C) + \#_{\mathrm{mm}}(C))$, there must be at least $2^k/(3\beta)$ aggregation gates.

For the upper bound, the explicit MATLANG expression (1) uses exactly 2^k matrix–vector aggregations; all remaining operations are entrywise or broadcasts. Its correctness for arbitrary semiring-valued inputs follows from distributivity, as verified in §7.1. Reusing partial products in the H_A as in §1.1 gives $O(2^k)$ total nodes. □

Theorem 7.4 (Boolean arity-elimination overhead)**.** *Over the Boolean semiring $\mathbb{B}$, every typed matrix circuit C computing G_k on all finite inputs satisfies $\#_{\mathrm{mv}}(C) + 2\#_{\mathrm{mm}}(C) \geq 2^k$. This holds with arbitrary uniform entrywise functions and aggregation kernels.*

Proof. The 2^k Boolean tests, Theorem 3.6, and the Boolean refinement of Lemma 7.2 give the claim. □

Why the Boolean case is not just a restatement of the CoR bound. Boolean matrix multiplication is relational composition, so total-size lower bounds for ordinary Boolean matrix expressions are closely related to Theorem 1.1. Here we count only aggregations: all other permitted matrix operations are free. For example, diagonal extraction followed by broadcasting obtains the matrix $(u, v) \mapsto R(u, u)$ without aggregation, whereas a CoR expression for it is $(R \cap \mathbf{id}); \mathbf{1}$, which uses a composition. Consequently, the aggregation count of a matrix circuit cannot be identified with the composition count of its CoR translation. We instead bound the fold sensitivity of the typed operations directly; that analysis also works over non-Boolean semirings.

Examples of breadth-one semirings include $\mathsf{Trop}_{\mathbb{N}}$, $\mathsf{Trop}_{\mathbb{Z}}$, the integer max–plus semiring $(\mathbb{Z} \cup \{-\infty\}, \max, +)$, and max–min on $[0, 1]$. All require at least $2^k/3$ aggregations, even in our enlarged kernel model.

7.5 Positive computations without idempotence

A separate transfer handles positive arithmetic, for which join breadth is not available. Let $(S, +, \otimes, 0_S, 1_S)$ be a nontrivial commutative semiring satisfying

$$r\,1_S \neq 0_S \qquad \text{for every integer } r \geq 1, \tag{NV}$$

where $r\,1_S$ is the sum of r copies of 1_S. Restrict the target to positive entrywise addition and multiplication, semiring matrix products, transpose, broadcasting, diagonal operations, and constants in $\mathbb{N}1_S := \{r\,1_S : r \geq 0\}$. Arbitrary entrywise decoding and negative constants are not allowed here. Replace joins in $\mathcal{G}_k$ by sums to define its positive semiring version.

On Boolean-encoded inputs, every intermediate belongs to $\mathbb{N}1_S$. The map sending 0_S to zero and every other element of this subsemiring to one is a homomorphism to the Boolean semiring. Indeed, sums and products of positive multiples of 1_S are again nonzero by the nonvanishing condition (NV). Applying this map to every gate of a target circuit C gives a Boolean matrix circuit with exactly the same typed product counts. Its output is G_k. The Boolean lower bound therefore gives $\#_{\mathrm{mv}}(C) + 2\#_{\mathrm{mm}}(C) \geq 2^k$. Here the cost is applied to the Booleanized computation; no argument by predictions is claimed for the original scalar circuit. The 2^k-product expansion still works by distributivity. This includes positive computations over $\mathbb{N}$ and nonnegative reals.

The restriction on the operations matters: with nonidempotent addition and arbitrary entrywise decoding, a single aggregation along each row can encode how many edges (x, z) have each profile of §4.3, and these counts determine G_k. The arbitrary-kernel result of Theorem 7.3 avoids this because finite join breadth limits how many colors can support a single aggregation.

References

[AB87] Noga Alon and Ravi B. Boppana. The monotone circuit complexity of Boolean functions. *Combinatorica*, 7(1):1–22, 1987. doi:10.1007/BF02579196.

[ADV+25] Josh Alman, Ran Duan, Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. More asymmetry yields faster matrix multiplication. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2005–2039. SIAM, 2025. doi:10.1137/1.9781611978322.63.

[AHS25] Mahmoud Abo Khamis, Xiao Hu, and Dan Suciu. Fast matrix multiplication meets the submodular width. *Proceedings of the ACM on Management of Data*, 3(2 (PODS)):98:1–98:26, 2025. doi:10.1145/3725235.

[AI03] Micah Adler and Neil Immerman. An $n!$ lower bound on formula size. *ACM Transactions on Computational Logic*, 4(3):296–314, 2003. doi:10.1145/772062.772064.

[All83] James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983. doi:10.1145/182.358434.

[Ass55] Günter Asser. Das Repräsentantenproblem im Prädikatenkalkül der ersten Stufe mit Identität. *Zeitschrift für Mathematische Logik und Grundlagen der Mathematik*, 1(4):252–263, 1955. doi:10.1002/malq.19550010403.

[ASU79a] Alfred V. Aho, Yehoshua Sagiv, and Jeffrey D. Ullman. Efficient optimization of a class of relational expressions. *ACM Transactions on Database Systems*, 4(4):435–454, 1979. doi:10.1145/320107.320112.

[ASU79b] Alfred V. Aho, Yehoshua Sagiv, and Jeffrey D. Ullman. Equivalences among relational expressions. *SIAM Journal on Computing*, 8(2):218–246, 1979. doi:10.1137/0208017.

[AYZ97] Noga Alon, Raphael Yuster, and Uri Zwick. Finding and counting given length cycles. *Algorithmica*, 17(3):209–223, 1997. doi:10.1007/BF02523189.

[BFK19] Karl Bringmann, Nick Fischer, and Marvin Künnemann. A fine-grained analogue of Schaefer's theorem in P: Dichotomy of $\exists^k\forall$-quantified first-order graph properties. In *34th Computational Complexity Conference (CCC)*, volume 137 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 31:1–31:27. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019. doi:10.4230/LIPIcs.CCC.2019.31.

[BFS86] László Babai, Peter Frankl, and Janos Simon. Complexity classes in communication complexity theory (preliminary version). In *27th Annual Symposium on Foundations of Computer Science (FOCS 1986)*, pages 337–347. IEEE Computer Society, 1986. doi:10.1109/SFCS.1986.15.

[BGG97] Egon Börger, Erich Grädel, and Yuri Gurevich. *The Classical Decision Problem*. Perspectives in Mathematical Logic. Springer, Berlin, Heidelberg, 1997. doi:10.1007/978-3-642-59207-2.

[BGV19] Robert Brijder, Marc Gyssens, and Jan Van den Bussche. On matrices and K-relations, 2019. arXiv:1904.03934v1.

[BGV22] Robert Brijder, Marc Gyssens, and Jan Van den Bussche. On matrices and K-relations. *Annals of Mathematics and Artificial Intelligence*, 90(2–3):181–210, 2022. doi:10.1007/s10472-021-09760-4.

[BGVW19] Robert Brijder, Floris Geerts, Jan Van den Bussche, and Timmy Weerwag. On the expressive power of query languages for matrices. *ACM Transactions on Database Systems*, 44(4):15:1–15:31, 2019. Conference version in ICDT 2018.

[BK99] Peter A. Boncz and Martin L. Kersten. MIL primitives for querying a fragmented world. *The VLDB Journal*, 8(2):101–119, 1999. doi:10.1007/s007780050076.

[BL21] Stanley Burris and Javier Legris. The algebra of logic tradition. The Stanford Encyclopedia of Philosophy, edited by Edward N. Zalta, 2021. Substantive revision of 12 February 2021. URL: https://plato.stanford.edu/entries/algebra-logic-tradition/.

[Bra00] Geraldine Brady. *From Peirce to Skolem: A Neglected Chapter in the History of Logic*, volume 4 of *Studies in the History and Philosophy of Mathematics*. North-Holland, Amsterdam, 2000.

[BS21] Karl Bringmann and Jasper Slusallek. Current algorithms for detecting subgraphs of bounded treewidth are probably optimal. In Nikhil Bansal, Emanuela Merelli, and James Worrell, editors, *48th International Colloquium on Automata, Languages, and Programming (ICALP 2021)*, volume 198 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 40:1–40:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPIcs.ICALP.2021.40.

[BVGS15] Paul Beame, Guy Van den Broeck, Eric Gribkoff, and Dan Suciu. Symmetric weighted first-order model counting. In *Proceedings of the 34th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS 2015)*, pages 313–328. ACM, 2015. doi:10.1145/2745754.2745760.

[CFI+24] Marco Carmosino, Ronald Fagin, Neil Immerman, Phokion G. Kolaitis, Jonathan Lenchner, and Rik Sengupta. Multi-structural games and beyond. *Logical Methods in Computer Science*, 20(4):27:1–27:40, 2024. arXiv:2301.13329. doi:10.46298/lmcs-20(4:27)2024.

[CH90] Kevin J. Compton and C. Ward Henson. A uniform method for proving lower bounds on the computational complexity of logical theories. *Annals of Pure and Applied Logic*, 48(1):1–79, 1990. doi:10.1016/0168-0072(90)90080-L.

[CM77] Ashok K. Chandra and Philip M. Merlin. Optimal implementation of conjunctive queries in relational data bases. In *Proceedings of the Ninth Annual ACM Symposium on Theory of Computing*, pages 77–90. ACM, 1977. doi:10.1145/800105.803397.

[CMW87] Isabel F. Cruz, Alberto O. Mendelzon, and Peter T. Wood. A graphical query language supporting recursion. In *Proceedings of the 1987 ACM SIGMOD International*

Conference on Management of Data, pages 323–330. ACM, 1987. doi:10.1145/38713.38749.

[Cod70] E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970. doi:10.1145/362384.362685.

[Cod72] E. F. Codd. Relational completeness of data base sublanguages. In Randall J. Rustin, editor, *Data Base Systems*, volume 6 of *Courant Computer Science Symposia*, pages 65–98. Prentice-Hall, Englewood Cliffs, NJ, 1972.

[Cop48] Irving M. Copilowish. Matrix development of the calculus of relations. *The Journal of Symbolic Logic*, 13(4):193–203, 1948. doi:10.2307/2267134.

[DHK20] Shaleen Deep, Xiao Hu, and Paraschos Koutris. Fast join project query evaluation using matrix multiplication. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1213–1223. ACM, 2020. doi:10.1145/3318464.3380607.

[DM56] Augustus De Morgan. On the syllogism, no. II. *Transactions of the Cambridge Philosophical Society*, 9:79–127, 1856. Read 25 February 1850.

[DM64] Augustus De Morgan. On the syllogism, no. IV, and on the logic of relations. *Transactions of the Cambridge Philosophical Society*, 10:331–358, 1864. Read 23 April 1860. Reprinted in Heath (ed.), *On the Syllogism and Other Logical Writings*, Routledge and Kegan Paul, 1966.

[EG04] Friedrich Eisenbrand and Fabrizio Grandoni. On the complexity of fixed parameter clique and dominating set. *Theoretical Computer Science*, 326(1–3):57–67, 2004. doi:10.1016/j.tcs.2004.05.009.

[Ehr61] Andrzej Ehrenfeucht. An application of games to the completeness problem for formalized theories. *Fundamenta Mathematicae*, 49(2):129–141, 1961. doi:10.4064/fm-49-2-129-141.

[ES13] William Ewald and Wilfried Sieg, editors. *David Hilbert's Lectures on the Foundations of Arithmetic and Logic 1917–1933*, volume 3 of *David Hilbert's Lectures on the Foundations of Mathematics and Physics, 1891–1933*. Springer, Berlin and Heidelberg, 2013. doi:10.1007/978-3-540-69444-1.

[EVW02] Kousha Etessami, Moshe Y. Vardi, and Thomas Wilke. First-order logic with two variables and unary temporal logic. *Information and Computation*, 179(2):279–295, 2002. doi:10.1006/inco.2001.2953.

[FGL+15] George H. L. Fletcher, Marc Gyssens, Dirk Leinders, Dimitri Surinx, Jan Van den Bussche, Dirk Van Gucht, Stijn Vansummeren, and Yuqing Wu. Relative expressive power of navigational querying on graphs. *Information Sciences*, 298:390–406, 2015. doi:10.1016/j.ins.2014.11.031.

[FLRV21] Ronald Fagin, Jonathan Lenchner, Kenneth W. Regan, and Nikhil Vyas. Multi-structural games and number of quantifiers. In *36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2021)*, pages 1–13. IEEE, 2021. Journal version: Logical Methods in Computer Science 21(1):10:1–10:37, 2025, doi:10.46298/lmcs-21(1:10)2025. doi:10.1109/LICS52264.2021.9470756.

[FM71] Michael J. Fischer and Albert R. Meyer. Boolean matrix multiplication and transitive closure. In *12th Annual Symposium on Switching and Automata Theory (SWAT 1971)*, pages 129–131. IEEE Computer Society, 1971.

[FR74] Michael J. Fischer and Michael O. Rabin. Super-exponential complexity of Presburger arithmetic. In Richard M. Karp, editor, *Complexity of Computation*, volume 7 of *SIAM–AMS Proceedings*, pages 27–41, Providence, RI, 1974. American Mathematical Society.

[FR79] Jeanne Ferrante and Charles W. Rackoff. *The Computational Complexity of Logical Theories*, volume 718 of *Lecture Notes in Mathematics*. Springer, 1979. doi:10.1007/BFb0062837.

[Fra54] Roland Fraïssé. Sur quelques classifications des systèmes de relations. *Publications Scientifiques de l'Université d'Alger, Série A*, 1:35–182, 1954.

[Fur70] M. E. Furman. Application of a method of fast multiplication of matrices in the problem of finding the transitive closure of a graph. *Soviet Mathematics Doklady*, 11(5):1252, 1970.

[FvdHIK13] Tim French, Wiebe van der Hoek, Petar Iliev, and Barteld Kooi. On the succinctness of some modal logics. *Artificial Intelligence*, 197:56–85, 2013. doi:10.1016/j.artint.2013.02.003.

[GI19] Jiawei Gao and Russell Impagliazzo. The fine-grained complexity of strengthenings of first-order logic. Report TR19-009, Electronic Colloquium on Computational Complexity, 2019. Revision 1, February 19, 2019. URL: https://eccc.weizmann.ac.il/report/2019/009/.

[Giv06] Steven Givant. The calculus of relations as a foundation for mathematics. *Journal of Automated Reasoning*, 37(4):277–322, 2006. doi:10.1007/s10817-006-9062-x.

[GM93] Zvi Galil and Oded Margalit. Witnesses for Boolean matrix multiplication and for transitive closure. *Journal of Complexity*, 9(2):201–221, 1993. doi:10.1006/jcom.1993.1014.

[GPW18] Mika Göös, Toniann Pitassi, and Thomas Watson. The landscape of communication complexity classes. *Computational Complexity*, 27(2):245–304, 2018. doi:10.1007/s00037-018-0166-6.

[GS05] Martin Grohe and Nicole Schweikardt. The succinctness of first-order logic on linear orders. *Logical Methods in Computer Science*, 1(1:6):1–25, 2005. Conference version in LICS 2004. doi:10.2168/LMCS-1(1:6)2005.

[GSSS87] Gösta Grahne, Seppo Sippu, and Eljas Soisalon-Soininen. Efficient evaluation for a subset of recursive queries (extended abstract). In *Proceedings of the 6th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS)*, pages 284–293. ACM, 1987. doi:10.1145/28659.28690.

[GSSS91] Gösta Grahne, Seppo Sippu, and Eljas Soisalon-Soininen. Efficient evaluation for a subset of recursive queries. *Journal of Logic Programming*, 10(3–4):301–332, 1991. doi:10.1016/0743-1066(91)90039-R.

[HA28] David Hilbert and Wilhelm Ackermann. *Grundzüge der theoretischen Logik*, volume 27 of *Die Grundlehren der mathematischen Wissenschaften*. Julius Springer, Berlin, 1928.

[Hås86] Johan Håstad. Almost optimal lower bounds for small depth circuits. In *Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing (STOC)*, pages 6–20. ACM, 1986. doi:10.1145/12130.12132.

[HPV+21] Jelle Hellings, Catherine L. Pilachowski, Dirk Van Gucht, Marc Gyssens, and Yuqing Wu. From relation algebra to semi-join algebra: An approach to graph query optimization. *The Computer Journal*, 64(5):789–811, 2021. doi:10.1093/comjnl/bxaa031.

[HSU74] Harry B. Hunt, III, Thomas G. Szymanski, and Jeffrey D. Ullman. Operations on sparse relations and efficient algorithms for grammar problems (extended abstract). In *Proceedings of the 15th Annual Symposium on Switching and Automata Theory (SWAT)*, pages 127–132. IEEE Computer Society, 1974. doi:10.1109/SWAT.1974.21.

[HSU77] Harry B. Hunt, III, Thomas G. Szymanski, and Jeffrey D. Ullman. Operations on sparse relations. *Communications of the ACM*, 20(3):171–176, 1977. doi:10.1145/359436.359446.

[Hu24] Xiao Hu. Fast matrix multiplication for query processing. *Proceedings of the ACM on Management of Data*, 2(2 (PODS)):98:1–98:25, 2024. doi:10.1145/3651599.

[HV19] Lauri Hella and Miikka Vilander. Formula size games for modal logic and μ-calculus. *Journal of Logic and Computation*, 29(8):1311–1344, 2019. doi:10.1093/logcom/exz025.

[HVGW22] Jelle Hellings, Dirk Van Gucht, Marc Gyssens, and Yuqing Wu. The power of Tarski's relation algebra on trees. *Journal of Logical and Algebraic Methods in Programming*, 126:100748, 2022. doi:10.1016/j.jlamp.2022.100748.

[HY20] Xiao Hu and Ke Yi. Parallel algorithms for sparse matrix multiplication and join-aggregate queries. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*, pages 411–425. ACM, 2020. doi:10.1145/3375395.3387657.

[IK89] Neil Immerman and Dexter Kozen. Definability with bounded number of bound variables. *Information and Computation*, 83(2):121–139, 1989. doi:10.1016/0890-5401(89)90055-2.

[Imm81] Neil Immerman. Number of quantifiers is better than number of tape cells. *Journal of Computer and System Sciences*, 22(3):384–406, 1981. doi:10.1016/0022-0000(81)90039-8.

[Imm82] Neil Immerman. Upper and lower bounds for first order expressibility. *Journal of Computer and System Sciences*, 25(1):76–98, 1982. doi:10.1016/0022-0000(82)90011-3.

[IR78] Alon Itai and Michael Rodeh. Finding a minimum circuit in a graph. *SIAM Journal on Computing*, 7(4):413–423, 1978. doi:10.1137/0207033.

[JMT95] Peter Jipsen, Roger D. Maddux, and Zsolt Tuza. Small representations of the relation algebra $E_{n+1}(1, 2, 3)$. *Algebra Universalis*, 33:136–139, 1995. doi:10.1007/BF01190770.

[JS74] Neil D. Jones and Alan L. Selman. Turing machines and the spectra of first-order formulas. *The Journal of Symbolic Logic*, 39(1):139–150, 1974. doi:10.2307/2272354.

[Juk12] Stasys Jukna. *Boolean Function Complexity: Advances and Frontiers*, volume 27 of *Algorithms and Combinatorics*. Springer, Berlin and Heidelberg, 2012. doi:10.1007/978-3-642-24508-4.

[KKM00] Ton Kloks, Dieter Kratsch, and Haiko Müller. Finding and counting small induced subgraphs efficiently. *Information Processing Letters*, 74(3–4):115–121, 2000. doi:10.1016/S0020-0190(00)00047-8.

[Kle52] Stephen Cole Kleene. *Introduction to Metamathematics*, volume 1 of *Bibliotheca Mathematica*. North-Holland and P. Noordhoff, Amsterdam and Groningen, 1952.

[KMW62] A. S. Kahr, Edward F. Moore, and Hao Wang. Entscheidungsproblem reduced to the $\forall\exists\forall$ case. *Proceedings of the National Academy of Sciences of the United States of America*, 48(3):365–377, 1962. doi:10.1073/pnas.48.3.365.

[KN97] Eyal Kushilevitz and Noam Nisan. *Communication Complexity*. Cambridge University Press, Cambridge, 1997. doi:10.1017/CBO9780511574948.

[KT15] Eryk Kopczyński and Tony Tan. On the variable hierarchy of first-order spectra. *ACM Transactions on Computational Logic*, 16(2):17:1–17:12, 2015. doi:10.1145/2733376.

[KT18] Eryk Kopczyński and Tony Tan. A note on first-order spectra with binary relations. *Logical Methods in Computer Science*, 14(2):4:1–4:15, 2018. doi:10.23638/LMCS-14(2:4)2018.

[KW90] Mauricio Karchmer and Avi Wigderson. Monotone circuits for connectivity require super-logarithmic depth. *SIAM Journal on Discrete Mathematics*, 3(2):255–265, 1990. doi:10.1137/0403021.

[Lev69] Roger E. Levien. Relational data file: Experience with a system for propositional data storage and inference execution. Technical Report RM-5947-PR, RAND Corporation, April 1969. URL: https://www.rand.org/pubs/research_memoranda/RM5947.html.

[Lib04] Leonid Libkin. *Elements of Finite Model Theory*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2004. doi:10.1007/978-3-662-07003-1.

[LM65] Roger E. Levien and M. E. Maron. Relational data file: A tool for mechanized inference execution and data retrieval. Technical Report RM-4793-PR, RAND Corporation, December 1965. URL: https://www.rand.org/pubs/research_memoranda/RM4793.html.

[LM67] Roger E. Levien and M. E. Maron. A computer system for inference execution and data retrieval. *Communications of the ACM*, 10(11):715–721, 1967. doi:10.1145/363790.363817.

[Löw15] Leopold Löwenheim. Über Möglichkeiten im Relativkalkül. *Mathematische Annalen*, 76(4):447–470, 1915. doi:10.1007/BF01458217.

[Löw67] Leopold Löwenheim. On possibilities in the calculus of relatives. In Jean van Heijenoort, editor, *From Frege to Gödel: A Source Book in Mathematical Logic, 1879–1931*, pages 228–251. Harvard University Press, Cambridge, MA, 1967. Translated by Stefan Bauer-Mengelberg from the 1915 German original.

[LS81] Richard J. Lipton and Robert Sedgewick. Lower bounds for VLSI. In *Proceedings of the Thirteenth Annual ACM Symposium on Theory of Computing*, pages 300–307. ACM, 1981. doi:10.1145/800076.802482.

[Mac81] Bruce J. MacLennan. Introduction to relational programming. In *Proceedings of the 1981 Conference on Functional Programming Languages and Computer Architecture (FPCA)*, pages 213–220. ACM, 1981. doi:10.1145/800223.806781.

[Mad91] Roger D. Maddux. The origin of relation algebras in the development and axiomatization of the calculus of relations. *Studia Logica*, 50(3–4):421–455, 1991. doi:10.1007/BF00370681.

[Mad06] Roger D. Maddux. *Relation Algebras*, volume 150 of *Studies in Logic and the Foundations of Mathematics*. Elsevier, Amsterdam, 2006.

[Mad21] Roger D. Maddux. Tarskian classical relevant logic. In Ivo Düntsch and Edwin Mares, editors, *Alasdair Urquhart on Nonclassical and Algebraic Logic and Complexity of Proofs*, volume 22 of *Outstanding Contributions to Logic*, pages 67–161. Springer, 2021. doi:10.1007/978-3-030-71430-7_3.

[Mon74] Ugo Montanari. Networks of constraints: Fundamental properties and applications to picture processing. *Information Sciences*, 7:95–132, 1974. doi:10.1016/0020-0255(74)90008-5.

[Moo88] Gregory H. Moore. The emergence of first-order logic. In William Aspray and Philip Kitcher, editors, *History and Philosophy of Modern Mathematics*, volume 11 of *Minnesota Studies in the Philosophy of Science*, pages 95–135. University of Minnesota Press, Minneapolis, 1988.

[Mor75] Michael Mortimer. On languages with two variables. *Zeitschrift für Mathematische Logik und Grundlagen der Mathematik*, 21(1):135–140, 1975. doi:10.1002/malq.19750210118.

[MS82] Kurt Mehlhorn and Erik M. Schmidt. Las Vegas is better than determinism in VLSI and distributed computing (extended abstract). In *Proceedings of the 14th Annual ACM Symposium on Theory of Computing (STOC 1982)*, pages 330–337. ACM, 1982. doi:10.1145/800070.802208.

[Mun71] Ian Munro. Efficient determination of the transitive closure of a directed graph. *Information Processing Letters*, 1(2):56–58, 1971.

[Nak19] Yoshiki Nakamura. The undecidability of FO^3 and the calculus of relations with just one binary relation. In Md. Aquil Khan and Amaldev Manuel, editors, *Logic and Its Applications*, pages 108–120. Springer, 2019. doi:10.1007/978-3-662-58771-3_11.

[Nak22] Yoshiki Nakamura. Expressive power and succinctness of the positive calculus of binary relations. *Journal of Logical and Algebraic Methods in Programming*, 127:100760, 2022. doi:10.1016/j.jlamp.2022.100760.

[NP85] Jaroslav Nešetřil and Svatopluk Poljak. On the complexity of the subgraph problem. *Commentationes Mathematicae Universitatis Carolinae*, 26(2):415–419, 1985. URL: https://dml.cz/handle/10338.dmlcz/106381.

[NU27] Yoshiki Nakamura and Yuya Uezato. A fine-grained dichotomy for bounded-variable query evaluation: The calculus of relations, a Boolean modal logic, and one-variable counting logic. In *46th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS 2027)*, 2027. To appear. Full version: arXiv:2610.04573.

[OO73] Patrick E. O'Neil and Elizabeth J. O'Neil. A fast expected time algorithm for Boolean matrix multiplication and transitive closure. *Information and Control*, 22(2):132–138, 1973. doi:10.1016/S0019-9958(73)90228-3.

[Pei70] Charles S. Peirce. Description of a notation for the logic of relatives, resulting from an amplification of the conceptions of Boole's calculus of logic. *Memoirs of the American Academy of Arts and Sciences*, 9:317–378, 1870.

[Pei80] Charles S. Peirce. On the algebra of logic. *American Journal of Mathematics*, 3(1):15–57, 1880. doi:10.2307/2369442.

[Pra92] Vaughan R. Pratt. Origins of the calculus of binary relations. In *Proceedings of the 7th Annual IEEE Symposium on Logic in Computer Science*, pages 248–254, Santa Cruz, CA, 1992. IEEE Computer Society. doi:10.1109/LICS.1992.185537.

[Raz85] Alexander A. Razborov. Lower bounds for the monotone complexity of some Boolean functions. *Soviet Mathematics Doklady*, 31(2):354–357, 1985. English translation of Doklady Akademii Nauk SSSR 281(4):798–801 (1985).

[RY20] Anup Rao and Amir Yehudayoff. *Communication Complexity and Applications*. Cambridge University Press, 2020. doi:10.1017/9781108671644.

[SAC+79] P. Griffiths Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, pages 23–34. ACM, 1979. doi:10.1145/582095.582099.

[Sav70] Walter J. Savitch. Relationships between nondeterministic and deterministic tape complexities. *Journal of Computer and System Sciences*, 4(2):177–192, 1970. doi:10.1016/S0022-0000(70)80006-X.

[Sch95] Ernst Schröder. *Vorlesungen über die Algebra der Logik (Exakte Logik), Dritter Band: Algebra und Logik der Relative*. B. G. Teubner, Leipzig, 1895.

[Sch89] Uwe Schöning. *Logic for Computer Scientists*, volume 8 of *Progress in Computer Science and Applied Logic*. Birkhäuser, Boston, 1989. English translation of *Logik für Informatiker*, B.I.-Wissenschaftsverlag, 1987.

[SM02] Larry J. Stockmeyer and Albert R. Meyer. Cosmological lower bound on the circuit complexity of a small problem in logic. *Journal of the ACM*, 49(6):753–784, 2002. Original result appeared in Stockmeyer's 1974 dissertation. doi:10.1145/602220.602223.

[SSS85] Seppo Sippu and Eljas Soisalon-Soininen. On the use of relational expressions in the design of efficient algorithms (extended abstract). In *Automata, Languages and Programming (ICALP 1985)*, volume 194 of *Lecture Notes in Computer Science*, pages 456–464. Springer, 1985. doi:10.1007/BFb0015771.

[Sto74] Larry J. Stockmeyer. *The Complexity of Decision Problems in Automata Theory and Logic*. PhD thesis, Massachusetts Institute of Technology, 1974. URL: https://dspace.mit.edu/entities/publication/2c1a238a-fd2f-4340-8751-097514b2f1fd.

[SU75] Thomas G. Szymanski and Jeffrey D. Ullman. Evaluating relational expressions with dense and sparse arguments. In *Proceedings of the 16th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 90–97. IEEE Computer Society, 1975. doi:10.1109/SFCS.1975.13.

[SU77] Thomas G. Szymanski and Jeffrey D. Ullman. Evaluating relational expressions with dense and sparse arguments. *SIAM Journal on Computing*, 6(1):109–122, 1977. doi:10.1137/0206009.

[Tar41] Alfred Tarski. On the calculus of relations. *The Journal of Symbolic Logic*, 6(3):73–89, 1941. doi:10.2307/2268577.

[TG87] Alfred Tarski and Steven Givant. *A Formalization of Set Theory without Variables*, volume 41 of *Colloquium Publications*. American Mathematical Society, Providence, RI, 1987. doi:10.1090/coll/041.

[Var95] Moshe Y. Vardi. On the complexity of bounded-variable queries. In *Proceedings of the Fourteenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS 1995)*, pages 266–276. ACM, 1995.

[VMD14] Guy Van den Broeck, Wannes Meert, and Adnan Darwiche. Skolemization for weighted first-order model counting. In *Proceedings of the 14th International Conference on Principles of Knowledge Representation and Reasoning (KR 2014)*. AAAI Press, 2014.

[Vor96] Sergei G. Vorobyov. An improved lower bound for the elementary theories of trees. In Michael A. McRobbie and John K. Slaney, editors, *Automated Deduction — CADE-13*, volume 1104 of *Lecture Notes in Computer Science*, pages 275–287. Springer, 1996. doi:10.1007/3-540-61511-3_91.

[VS24] Harry Vinall-Smeeth. From quantifier depth to quantifier number: Separating structures with k variables. In *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 71:1–71:14. ACM, 2024. Full version: arXiv:2311.15885. URL: https://arxiv.org/abs/2311.15885, doi:10.1145/3661814.3662125.

[Wil14] Ryan Williams. Faster decision of first-order graph properties. In *Joint Meeting of the Twenty-Third EACSL Annual Conference on Computer Science Logic and the Twenty-Ninth Annual ACM/IEEE Symposium on Logic in Computer Science (CSL-LICS)*, pages 80:1–80:6. ACM, 2014. doi:10.1145/2603088.2603121.

[Yan81] Mihalis Yannakakis. Algorithms for acyclic database schemes. In *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB)*, pages 82–94. IEEE Computer Society, 1981.

[Yao79] Andrew Chi-Chih Yao. Some complexity questions related to distributive computing (preliminary report). In *Proceedings of the 11th Annual ACM Symposium on Theory of Computing (STOC 1979)*, pages 209–213. ACM, 1979. doi:10.1145/800135.804414.

[YZ04] Raphael Yuster and Uri Zwick. Detecting short directed cycles using rectangular matrix multiplication and dynamic programming. In *Proceedings of the Fifteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 247–253. SIAM, 2004.

A Explicit two-witness colorings

This appendix gives a deterministic version of Lemma 3.1 on selected domain sizes. The affine-plane construction follows the direction-partition method of Jipsen, Maddux, and Tuza [JMT95]. Their assignment of at least two directions to every color already guarantees two witnesses at plane order at least four; we include the verification because this is the property used here.

Proposition A.1 (Explicit two-witness colorings)**.** *For every palette size $N \geq 2$, a graph with the two-witness property of Lemma 3.1 can be constructed deterministically on fewer than $16N^2$ vertices.*

Proof. Let s be the smallest power of two with $s \geq 2N - 1$, and let $\mathbb{F}_s$ be the field of s elements. Use vertex set $V := \mathbb{F}_s^2$. There are $s + 1$ directions of affine lines. Partition these directions into sets Dir_μ, $\mu \in \Lambda$, each of size at least two. Color distinct points u, v by the unique μ containing the direction of their joining line. Since $N \geq 2$, we have $s \geq 4$ and $|V| = s^2 < 16N^2$.

Fix distinct u, v and requested colors μ, ν. If $\mu = \nu = \chi(u, v)$, all $s - 2 \geq 2$ other points of the line uv are witnesses. Otherwise, choose a direction from Dir_μ at u and a direction from Dir_ν at v, neither equal to the direction of uv, and choose the two directions to be distinct. Their two lines

intersect at a unique witness outside $\{u, v\}$. There are at least two choices: if the colors differ, their direction sets are disjoint and at most one set loses a direction to uv; if they agree but differ from $\chi(u, v)$, two directions from their common set can be taken in either order. Distinct direction pairs give distinct intersection points, since a point outside $\{u, v\}$ determines both directions uniquely. Thus the coloring has the two-witness property. □

For $N = 2^k$, one can choose $s = 2N$, giving 4^{k+1} vertices. This construction supplies selected domain sizes, rather than every large size; induced subgraphs need not retain the two-witness property.

B Tree bounds and limits of the method

B.1 The input-occurrence charging lemma

This subsection proves the tree refinement stated in §6.2.

Lemma 6.2 (Disjoint input-occurrence charging)**.** *Fix the reference structure and its folds. Suppose every singleton selector requires at least $d > 0$ input-relation occurrences. Then every* CoR *term T, whether correct or not, satisfies*

$$\mathrm{cost}(T) \le \#_{\mathrm{rel}}(T)/d.$$

Proof. For a subterm whose value on the reference structure is X, let $S_X \subseteq \Lambda$ be the off-diagonal support of X. Boolean operations act on these sets in the usual way and converse fixes them. By the identities (CT), a composition $X; Y$ with both supports nonempty has off-diagonal support Λ. Call such a node *active*. If either support is empty, the off-diagonal table of the product is zero or one of the two operand tables, according to the diagonal bits. Such an inactive node notices no fold: every outside-witness contribution in the local calculation vanishes.

Construct a Boolean formula for the off-diagonal table of any subterm by replacing active compositions by the constant true, inactive compositions by zero or the relevant operand table, and converse by its argument. This operation never duplicates an input occurrence. For each active node $g = (X; Y)$, apply this pruning to its two operands. Let $\mathcal{L}_g$ be the set of original input occurrences that remain in the two resulting table formulas. These are the occurrences that *survive the pruning*, not all input occurrences below g. This distinction prevents repeated charging at successive active nodes. The sets $\mathcal{L}_g$ are pairwise disjoint: an active descendant is replaced by a constant when an ancestor's operands are pruned, and incomparable nodes have disjoint subtrees.

By Lemma 3.5, a composition notices the fold of a color λ only if

$$S_X = \{\lambda\}, \qquad S_Y = \{\lambda\}, \qquad \text{or} \qquad S_X \cap S_Y = \{\lambda\}.$$

Let E_g be the set of colors satisfying at least one of these conditions. Thus $|D_g| \le |E_g|$. If $S_X \cap S_Y$ is nonempty, E_g has size at most one. Its unique color, if present, is selected by one operand table or by their conjunction. The selector assumption therefore gives $|\mathcal{L}_g| \ge d|E_g|$. If the two supports are disjoint, only singleton operands contribute to E_g. Each contributing operand has at least d input occurrences, and the operands are disjoint trees. The same inequality follows. Summing over active nodes proves

$$\mathrm{cost}(T) = \sum_g |D_g| \le \sum_g |E_g| \le \frac{1}{d} \sum_g |\mathcal{L}_g| \le \#_{\mathrm{rel}}(T)/d.$$

Diagonal bits may affect which operand table an inactive node retains, but do not create additional input occurrences in this argument. □

B.2 Factored trees for G_k

A complete binary decision tree successively chooses P_i or Q_i for the read edge. At leaf A place $H_{A^c}; \mathbf{1}$. A branch at coordinate i with child terms U_1, U_0 has the form

$$(P_i \cap U_1) \cup (Q_i \cap U_0).$$

Distributing the branches recovers the DNF for G_k on arbitrary inputs; no assumption that exactly one of P_i, Q_i holds at each edge is used. Each of the 2^k leaves costs $2k+1$ nodes, and each of the $2^k - 1$ internal branches adds five. The total is $(2k+6)2^k - 5$. The number of relation-input occurrences is $k2^k + 2(2^k - 1)$.

B.3 Why the requirements are crossed

For comparison, the superficially similar non-crossed formula collapses:

$$\exists z \bigwedge_{i=1}^{k} [(P_i(x,y) \wedge P_i(x,z)) \vee (Q_i(x,y) \wedge Q_i(x,z))] \equiv \bigwedge_{i=1}^{k} (P_i(x,y) \vee Q_i(x,y)).$$

The forward implication is immediate; for the reverse implication take $z = y$. In G_k, however, the two edges are subject to crossed requirements, so choosing $z = y$ need not satisfy them.

B.4 Sharpness of the fold tests

Proposition B.1 (Sharpness on the folded structures)**.** *For every $k \geq 1$, the minimum number of composition gates in a relational circuit that computes G_k on the reference structure and on all its folded structures is 2^{k-1}.*

Proof. For each unordered complementary pair $\{A, A^c\}$ of colors, choose one representative A and compute, once,

$$X_A \coloneqq H_A; H_{A^c}.$$

Use the shared result in

$$\bigcup_{\{A,A^c\}} \left[(H_{A^c} \cap X_A) \cup (H_A \cap X_A^{\smile})\right].$$

This circuit has exactly 2^{k-1} compositions. We check that it computes G_k on $\mathcal{M}_0$ and on every $\mathcal{M}_\lambda$. Let J be the set of colors at the hub, so that $J = \Lambda$ on $\mathcal{M}_0$ and $J = \Lambda \setminus \{\lambda\}$ on $\mathcal{M}_\lambda$. On each of these structures, H_A is the set of edges of color A. The relation X_A is true on all pairs of distinct non-hub vertices, has zero diagonal, and has hub-row and hub-column values $[A \in J]$ and $[A^c \in J]$, respectively. The masks select the complementary read colors. At a non-hub vertex every color still occurs, so G_k is true on every off-diagonal read edge there. At the hub it is true precisely when the complementary color lies in J. These are exactly the displayed circuit's values. The lower bound is Theorem 1.1, whose proof uses only these structures. □

C Circuits without converse

We prove the sharper bound for circuits without converse with a directed version of the reference structure.

Corollary 4.3 (Without converse)**.** *For every $k \geq 1$, every relational circuit without converse that defines G_k on finite structures has at least 2^k composition gates. This bound is attained by the DNF.*

Proof. Color each ordered pair of distinct vertices independently from the palette $\Lambda = 2^{[k]}$; write $\chi(u, v)$ for its color, with no relation between $\chi(u, v)$ and $\chi(v, u)$. Require that for every u, v, now allowing $u = v$, and every $\mu, \nu \in \Lambda$, at least two vertices z outside $\{u, v\}$ realize

$$(\chi(u, z), \chi(z, v)) = (\mu, \nu).$$

Candidate witnesses use disjoint ordered arcs, even when $u = v$, so the proof of Lemma 3.1 applies with the same threshold $n_0(N)$ and at most n^3N^2 bad events. Interpret P_i and Q_i on each arc by its color as in §4.1, with all loop entries zero.

Reference relations have a diagonal value $X[\mathbf{id}]$ and one value $X[\mu]$ per arc color μ; let S_X be the set of colors on which X holds. These tables are closed under Boolean operations and composition: endpoint witnesses contribute as in (CT), and every pair, including a diagonal pair, has an outside witness exactly when S_X and S_Y are both nonempty. The fold of λ now recolors only the arcs of color λ that leave $\hbar$, to λ^c, and leaves all arcs entering $\hbar$ unchanged; predictions read the tables on this new coloring. Outside the row of $\hbar$, the composition of the predicted operands equals the prediction of the composition, since for every pair of colors some outside witness other than $\hbar$ uses two unchanged arcs. In the row of $\hbar$, including $(\hbar, \hbar)$, an outside witness exists exactly when $S_X \setminus \{\lambda\}$ and S_Y are both nonempty. Hence a composition notices the fold of λ only if $S_X = \{\lambda\}$, and so it notices at most one fold.

Choose v_μ so that the arc $(\hbar, v_\mu)$ has color μ, and read the output at $(\hbar, v_{\lambda^c})$. The query G_k inspects only arcs leaving x, so the first assertion of Lemma 4.1 holds with the same proof, reading the edges at a vertex as the arcs leaving it, and Theorem 3.6 gives $g_{;}(C) \geq 2^k$ for relational circuits without converse. The DNF of G_k uses no converse and attains this bound on arbitrary inputs. Converse is excluded because the color of an arc does not determine the color of its reverse arc; the coefficient-one argument consequently does not apply to full CoR. □

D The classical single-exponential translation

We include the size analysis for the classical expressive equivalence [TG87, Mad06]. The relevant point is that successive quantifier eliminations need not create a tower of exponentials: an already translated subformula is treated as one leaf of the next Boolean expansion, without expanding its algebraic representation again.

Proposition D.1 (Classical translation)**.** *Every* FO^3 *formula φ over binary relations with at most two free variables has an equivalent* CoR *term of size $2^{O(|\varphi|)}$ on all nonempty structures, constructible in time $2^{O(|\varphi|)}$. Here size counts syntax-tree nodes.*

Proof. We use the binary-assignment convention of §2.1: represent each subformula with at most two free variables on an ordered pair containing them. An unused coordinate is a dummy, so moving

the representation to another suitable pair requires only converse (if the coordinates are exchanged). A sentence is represented by **0** or **1**.

The atomic translations are those of §2.2, with the two relevant variables renamed or reordered: distinct-variable atoms use the input relation or its converse, reflexive atoms use a diagonal test followed by a projection, and equality uses **id** or **1**. All have constant size. Boolean operations on a fixed variable pair become intersection, union, and complement.

Process quantified subformulas from the inside out. Consider $\exists w\, \psi$, and let u, v be the other two variable names, in that order. Its free variables are contained in $\{u, v\}$. In the body ψ, regard each atom and each maximal quantified subformula as one leaf of a Boolean formula, called its *shell*. Each leaf has at most two free variables: this is immediate for binary atoms, and a quantifier binds one of the three available names. The quantified leaves have already been translated. Crucially, the shell expansion treats each of their translations as an opaque term.

If the shell has r leaf occurrences, a truth-table DNF has at most 2^r clauses, each using at most r literals. Repeated or logically dependent leaves cause no problem; the propositional identity remains valid after substituting their actual interpretations. In each clause, assign its literals to three groups according to the free-variable pair: (u, v), (u, w), or (w, v). A literal with only one free variable is assigned to any group containing it; a closed literal can be put in the first group. Use converse to orient each translated literal for its chosen pair, and use complement for a negative literal. Writing the conjunctions of the groups as $A(u, v)$, $B(u, w)$, and $C(w, v)$, apply the identity (QE):

$$\exists w\, (A(u, v) \wedge B(u, w) \wedge C(w, v)) \quad \longmapsto \quad A \cap (B; C).$$

An empty group is represented by **1**. This identity also covers vacuous quantification because domains are nonempty. Take the union of the translated clauses. Universal quantification uses $\forall w\, \psi \equiv \neg\exists w\, \neg\psi$, which introduces no subformula duplication. At the root, any remaining Boolean shell with at most two free variables is translated directly, after orienting its leaves to a common ordered pair. This proves correctness on all nonempty structures by induction.

For the size bound, cut the original syntax tree at quantifiers into the shells just used. Count each child quantified subformula as one leaf of its parent's shell. The sum of the shell sizes, including these boundary leaves, is $O(|\varphi|)$: each original node is counted once, with only one extra boundary occurrence per quantified subformula. A shell of size s creates at most $2^{O(s)}$ local nodes and at most $2^{O(s)}$ copies of each child translation. Along any path from an original leaf or a newly introduced local node to the final output, the resulting duplication factor is therefore at most

$$\prod_{\text{shells on the path}} 2^{O(s)} \leq 2^{O(|\varphi|)}.$$

There are $O(|\varphi|)$ original nodes and shell locations, and the local exponential creation factor is included in this same bound. Summing their contributions gives $|\varphi|\, 2^{O(|\varphi|)} = 2^{O(|\varphi|)}$ nodes. No step expands the internal Boolean structure of a translated child, which is what prevents an iterated-exponential recurrence. Enumerating the shell assignments and writing the resulting term takes a polynomial factor in $|\varphi|$ times the same exponential bound, hence also $2^{O(|\varphi|)}$ time. □